

MixBytes()

[SECURITY AUDIT REPORT]

Shift DeFi Platform

PREPARED FOR SHIFT DEFI

REPORT DATE

July 31, 2026

Table of Contents

1. Introduction	5
1.1 Disclaimer	5
1.2 Executive Summary	5
1.3 Project Overview	8
1.4 Security Audit Methodology	11
1.5 Risk Classification	13
1.6 Summary of Findings	14
2. Findings Report	19
2.1 Critical	19
C-1 Reshuffling Gateway Assets Can Be Drained via Share Transfer	19
2.2 High	20
H-1 Strategy and Vault Can Be Bricked due to State Desync in Emergency Exit	20
H-2 Unauthorized Claim on Behalf of Users Leads to Fund Loss in Repairing Mode	22
H-3 Vault DoS When Container Is Added during Active Batch	23
H-4 Users Lose Rights to Future Recovered Assets After First Claim in Repairing Mode	24
H-5 Inconsistent Claim Logic between claimDeposit() and claimReshufflingGateway() Leads to Fund Loss	25
H-6 Missing Binding between Burn and Receiver Messages Can Lock or Miscredit Funds	27
H-7 Buffered Deposits and Pending Withdrawals Locked in Repairing Mode	28
H-8 Inconsistent Bridge Return Amount Breaks Cross-Chain Claims	29
H-9 Withdrawal Batch BPS Truncation Causes Up to ~50% Loss for Withdrawers Near Basis-Point Boundaries	30
H-10 Unclaimed Withdrawal Shares Corrupt totalSupply-Based Calculations	31
H-11 Share Price Inflation Attack	33
2.3 Medium	34
M-1 Missing notInRepairingMode Modifier in withdraw() Function	34
M-2 Unsafe Token Approval Across Multiple Contracts	35
M-3 Missing handleReceiveUnfinalizedMessage() Callback Can Cause Claim Reverts	36
M-4 Withdrawal Batch Processing Can Be Indefinitely Stuck Due to minWithdrawBatchRatio Threshold	37
M-5 depositWithPermit() Is Front-Runnable	38

M-6 lastResolvedDepositBatchBlock and lastResolvedWithdrawBatchBlock Are Never Updated After Initialization	39
2.4 Low	40
L-1 CustomPool.swap() Transfers Tokens from receiver instead of msg.sender	40
L-2 Fee Cap Validation Uses Strict Inequality (<) Instead of Less Than or Equal (<=)	41
L-3 Insufficient Cache Size for Bridge Retries	42
L-4 UniversalAdapter Transfers Full Balance Instead of Swap Delta	43
L-5 Ambiguous Cache Key for Identical Bridge Transactions	44
L-6 Deprecated Chainlink latestAnswer Function	45
L-7 Unused Error Definitions	46
L-8 Missing Support for LayerZero Token Fee Payments in MessageRouter	47
L-9 Incorrect Strict Inequality in Slippage Check Prevents Exact Match Success	48
L-10 Redundant Caching of Received Messages Reduces Retry Window	49
L-11 Protocol DoS via skipDepositBatch in Empty Vault	50
L-12 ShiftOApp Refund Address Uses tx.origin	51
L-13 RingCacheLibrary Defines Unused DEFAULT_CACHE_SIZE Constant	52
L-14 Anyone Can Grief Container Deposit Reporting by Transferring Whitelisted Tokens	53
L-15 Strategy Exit and Enter Uses Pull Pattern for Transferring Tokens, Reducing Composability	54
L-16 CCTPv2 blacklistDomain Does Not Validate domainId Matches Stored Value and Leaks Stale Reverse Mappings	55
L-17 AcrossBridgeAdapter Validates fillDeadline But Never Uses It in the Bridge Call	56
L-18 UniswapV3Adapter whitelistPath Allows Zero-Hop Paths	57
L-19 Naming and Error Message Clarity	58
L-20 Redundant Unchecked Loop Increments	59
L-21 Vault claim* Reverts When Nothing to Claim, Breaking Integrations	60
L-22 Meaningless Balance-Delta Check After Bridging	61
L-23 Too Strict Slippage Check on bridgedAmount	62
L-24 Nonce Check Can Break Delivery on Message Reordering	63
L-25 No Way to Remove Predefined Swaps	64
L-26 Strict Inequalities in Deposit Limits	65
L-27 setContainerWeights() Allows Duplicate Container Addresses	66
L-28 startDepositBatchProcessing Can Revert on Zero Container Amount	67
L-29 Single EMERGENCY_PAUSER_ROLE Controls Both pause() and unpause() With No Threshold Differentiation	68

L-30 Missing <code>setConfig()</code> Wrapper for Library Parameter Tuning	69
L-31 Missing <code>_disableInitializers()</code>	70
L-32 <code>blacklistDomain</code> Leaves Stale <code>domainId</code> in Domain Storage	71
L-33 <code>CustomOracleWrapper.getPrice()</code> — Returns Zero for Unset Tokens Without Reverting	72
L-34 <code>ChainlinkOracleWrapper</code> — Single Global Staleness Threshold for All Feeds	73
L-35 <code>ChainlinkOracleWrapper</code> — No L2 Sequencer Uptime Check	74
L-36 <code>PriceOracleAggregator</code> — Single Oracle per Token With No Fallback	75
L-37 <code>Container</code> , <code>CrossChainContainer</code> , <code>StrategyTemplate</code> — Missing Storage Gaps (<code>__gap</code>) in Base Abstract Contracts	76
L-38 <code>CrossChainContainer.setBridgeAdapter()</code> and <code>ReshufflingGateway.blacklistBridgeAdapter()</code> — De-Listed Adapters Retain Token Approvals	77
L-39 <code>StrategyContainer.isStrategyNavUnresolved()</code> — Incorrect Result for Non-Existent Strategies	78
L-40 <code>Vault.claimWithdraw()</code> — Shares Permanently Locked When Notion Rounds to Zero	79
L-41 Empty <code>DepositRequest</code> Leaves <code>ContainerAgent</code> in Unrecoverable <code>DepositRequestReceived</code>	80
L-42 Strict Slippage Inequality in <code>_enterToState()</code> Reverts on Exact <code>minNavDelta</code> Match	81
L-43 <code>defaultPriceFeedStalenessThreshold</code> Has No Post-Construction Setter in <code>ChainlinkOracleWrapper</code>	82
L-44 <code>StrategyTemplate.emergencyExit()</code> — Failure Handling Is Unreachable Unless <code>emergencyExitMaxSlippage == MAX_BPS</code>	83
L-45 <code>StrategyTemplate</code> — Dangling Router Allowance After a No-Route Swap	84
L-46 <code>ShiftOApp</code> Ordered Delivery Omits the <code>skipInboundNonce</code> Recovery Helper	85
3. About MixBytes	86

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Shift DeFi platform is a cross-chain yield aggregator where users deposit a notion token into a share-based Vault to earn yield and receive shares upon deposit. Deposits and withdrawals are processed in batches across multiple containers. Each container allocates and redeems funds in different strategies, either on its local chain or across blockchains. Cross-chain messaging and token transfers are performed through LayerZero OApp, CCTP v2, and Across bridges. The platform also integrates swap adapters that interface with protocols such as Uniswap V3 and 1inch for token swaps.

The interim audit took 20 days and was carried out by 3 auditors using manual code review and static analysis. All re-audit stages are documented in detail in the Versions Log.

In this audit, we focused on attack scenarios targeting the core protocol logic as well as the project's interactions with its integrations. In addition, we applied our internal checklist covering various attack vectors such as reentrancy, access control, integer overflows, rounding errors, and others.

Strategy implementations that are direct inheritors of `StrategyTemplate` were out of scope for this audit. We recommend that they be audited separately and that their compatibility with in-scope contracts be explicitly verified, including integration tests, access control assumptions, and handling of edge cases.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Overall Assessment.** The codebase is structured and the main deposit, withdrawal, and cross-chain flows are generally well implemented. A large share of the findings, however, relate to Repairing Mode and emergency or edge-case paths. We recommend revisiting the design of Repairing Mode and related recovery flows, tightening invariants and access control there, and adding tests and documentation for those scenarios. By the time of the reaudit, the Repairing Mode design had been revisited architecturally and the test suite had been improved.
- **Notion Token.** Accounting in the Vault assumes that the amount transferred equals the amount received. With fee-on-transfer or rebasing tokens, this invariant breaks. The protocol is intended to be used with standard ERC-20 notion tokens (no transfer hooks, fee-on-transfer, or rebasing).
- **Bridge Adapters.** The project integrates multiple bridge adapters. Several issues related to the bridge adapters were identified by the audit team. Besides fixing these issues, we strongly recommend that the development team ensure they are fully familiar with the logic of each bridge they integrate. A complete understanding of how each bridge works—message formats, attestation flow, finality, and edge cases—is essential to avoid integration bugs.
- **LayerZero DVNs.** The security of each production LayerZero route depends directly on the selected DVN configuration. We recommend using multiple independent DVNs per route, rather than relying on a single verifier or a tightly concentrated verifier set.
- **CustomPool.** `CustomPool` is a swap adapter that quotes at a protocol-set exchange rate when there is no liquid market on a chain. The pool must be pre-funded with the output token. It does not provide liquidity itself and only applies the configured rate to existing reserves. The exchange rate is set by `TOKEN_MANAGER_ROLE` with no on-chain bounds, so the token manager can set an arbitrary rate and drain the pre-funded balance. The audit assumes the token manager is trusted and that pre-funded amounts are acceptable given this trust model.

- **Bridge Adapter Refunds and `retryBridge`.** In adapters such as Across, failed bridge operations can result in refunds to the depositor. Because the adapter both receives and sends funds, unclaimed incoming balances can mix with refunds. A mistaken privileged `BridgeAdapter.retryBridge()` call can then irreversibly re-bridge unrelated funds. We recommend ensuring that `BridgeAdapter.retryBridge()` is only invoked when the adapter balance is unambiguous, or considering separation of refund and incoming flows to avoid this risk.
- **Repairing Mode.** Repairing Mode is currently associated with a large number of findings in this audit. We recommend that the team revisit the design and behavior of Repairing Mode: simplify the state transitions where possible, tighten access and invariants, and ensure that user recovery paths and operator responsibilities are clearly defined and tested. By the time of the reaudit, Repairing Mode had been revisited architecturally: the related logic had been removed from the codebase, and all findings in this report that depended on Repairing Mode had been fixed.
- **Asynchronous Share Minting.** The protocol mints user shares asynchronously: the user deposits funds but does not receive shares immediately and can only claim them after some time. During this interval the user does not hold shares yet and is exposed—if something goes wrong (e.g. protocol failure, bridge or container issue), they may incur significant losses. It may be worth considering an architecture where shares are minted synchronously at deposit time (e.g. using external oracles to determine share allocation immediately).
- **Roles and Access Control.** The project uses a role-based access control system. The operator role currently has broad decision-making power and can call many different functions across the protocol. We recommend considering a dedicated role for executing swaps on containers, so that swap-related permissions can be scoped and the operator's blast radius is reduced. We also recommend paying close attention to whom each role is granted: carefully vet addresses before assigning roles and follow best practices such as using multisig for privileged roles to reduce single points of failure. A two-step (propose/accept) process for setting critical parameters can reduce both the risk of mistakes and malicious actions. A timelock for critical parameter changes may help mitigate the risks associated with centralization by giving users and the community time to react before changes take effect.
- **LayerZero EID-ChainId Mapping Integrity.** `Shift0App.setEidAndChainId()` does not validate whether a mapping for the given EID or chain ID already exists. If called twice with conflicting values for the same chain ID, the bidirectional mapping becomes inconsistent: `eidToChainId` retains the stale entry for the old EID while `chainIdToEid` is overwritten with the new one. As a result, `_validateEid()` will reject the old EID, causing `setSendLibrary` / `setReceiveLibrary` calls for that EID to revert. In practice this requires an operator misconfiguration, since LayerZero assigns each chain a single immutable EID. We recommend either preventing overwriting existing entries (requiring explicit removal first) or automatically clearing stale reverse-mapping entries on update.
- **Role Revocation Across Independently Deployed Contracts.** The same logical role names (`OPERATOR_ROLE` , `RESHUFFLING_EXECUTOR_ROLE` , and similar) appear on multiple contracts that are deployed separately, each with its own `AccessControl` / `AccessControlUpgradeable` storage. Revoking or rotating a role on one contract does not affect the others: an address removed from one deployment may still act on another. That is mainly an operational and governance risk, not a defect in any single contract's access checks. We recommend maintaining a written procedure for role changes that lists every contract instance and requires synchronized `revokeRole` / `grantRole` updates, or, if stronger guarantees are desired, a centralized role source (registry or shared module) so policy is not split across unrelated deployments.
- **StrategyTemplate Re-review and LayerZero `nextNonce` Update.** A follow-up review of the shared `StrategyTemplate` base (platform commit `6863b8a`) surfaced low-severity, defense-in-depth issues in the emergency-exit and swap-allowance paths, added as Low #44–#45. Neither relates to the changes introduced in the changes; both are pre-existing properties of the `StrategyTemplate` base surfaced during the full re-review. Separately, the LayerZero adapter's `nextNonce` change (lz-v2 commit `c28061e`) correctly hardens delivery ordering on-chain — it enforces gap-free per-channel nonces via `nextNonce` / `_acceptNonce` , complementing the `addExecutorOrderedExecutionOption` the adapter already used — but, matching the canonical LZ OApp example, it omits the `skipInboundNonce` recovery helper the LZ docs prescribe. Since the synchronous receive path (`MessageRouter` → `ContainerPrincipal` / `ContainerAgent.receiveMessage`) reverts when the target container is paused, in emergency/reshuffling, or off-status, a head-of-line revert stalls the whole shared channel; transient reverts self-heal on retry, and a would-be permanent revert is cleared by upgrading the (upgradeable) `MessageRouter` / `Container` so the message processes — a low-severity robustness gap. This is added as Low #46. The latest remediation round addressed all newly reported low-

severity findings from the StrategyTemplate and LayerZero follow-up review. The reviewed remediation diff was also checked for regressions, and no new reportable vulnerabilities were identified.

1.3 Project Overview

Summary

CLIENT	Shift
CATEGORY	Yield Aggregator
PROJECT	DeFi Platform
TYPE	Solidity
PLATFORM	EVM
TIMELINE	12.01.2026 - 08.07.2026

Scope of Audit

contracts/libraries/helpers/Errors.sol

contracts/libraries/helpers/EnumerableAddressSetExtended.sol

contracts/libraries/helpers/RingCacheLibrary.sol

contracts/libraries/helpers/Common.sol

contracts/libraries/StrategyStateLib.sol

contracts/libraries/Codec.sol

contracts/priceOracles/ChainlinkOracleWrapper.sol

contracts/priceOracles/CustomOracleWrapper.sol

contracts/SwapRouter.sol

contracts/ContainerLocal.sol

contracts/MessageRouter.sol

contracts/Vault.sol

contracts/BridgeAdapter.sol

contracts/StrategyContainer.sol

contracts/ReshufflingGateway.sol

contracts/ContainerPrincipal.sol

contracts/ContainerAgent.sol

contracts/PriceOracleAggregator.sol

contracts/Container.sol

contracts/StrategyTemplate.sol

contracts/CrossChainContainer.sol

contracts/UniswapV3Adapter.sol

contracts/UniversalAdapter.sol

contracts/CustomPool.sol

contracts/OneInchAdapter.sol

contracts/Shift0App.sol

contracts/CCTPv2BridgeAdapter.sol

contracts/AcrossBridgeAdapter.sol

contracts/libraries/Common.sol

contracts/libraries/EnumerableAddressSetExtended.sol

contracts/libraries/Errors.sol

contracts/libraries/RingCacheLib.sol

Versions Log

DATE	COMMIT HASH	NOTE
12.01.2026	97902065282e09e938698b88b601d1c0a04914db	Initial Commit (platform)
12.01.2026	219071983405befbe221fe3f57e5c33ee29fc770	Initial Commit (swap-adapters)
12.01.2026	0606e3feab30b4d3973056ca35edbc35f43955c9	Initial Commit (lz-v2)
12.01.2026	c3981ad0c9400c9abb8e11d0a418a87494ba643e	Initial Commit (cctp-v2)
12.01.2026	f40ddaca4215dd4acf7006b72144c8b94a047b1f	Initial Commit (across-v3)
26.03.2026	98f9ff21f9d2bf893ec47ddce148cfb0f3891ef2	Commit for re-audit (lz-v2)
26.03.2026	85d41f98ac86322d3576c18a92b1a9176b5b67cb	Commit for re-audit (cctp-v2)
26.03.2026	59c80e131c023eb7c9dd6fef26e67622d46d3bba	Commit for re-audit (across-v3)
26.03.2026	3e47d624d1c8084544d16c40622e5dc6666c9e76	Commit for re-audit (swap-adapters)
26.03.2026	f3db81e97346ad5a6fb7846ed934e5fb03d8a105	Commit for re-audit (platform)
03.04.2026	8392c1b37650ad2a8f64f046742573057e8d6002	Commit for re-audit (platform)
17.04.2026	79ca6dc8623bbdd1a4aa9d616a1a4514f9799b72	Commit for 2nd re-audit (platform)
17.04.2026	fe689dbda59d9c8865cb719cd76fff72700881fcf	Commit for 2nd re-audit (lz-v2)
17.04.2026	a9c4897463bd62763f562ea581cc3ebf903f34dc	Commit for 2nd re-audit (cctp-v2)
17.04.2026	241d66710ccd005fa7c11b0b3390f849f9b0bb5f	Commit for 2nd re-audit (across-v3)
01.05.2026	c9d848e18ef820c3ae093e6b6af6b2cf220aac57	Low-43 fix commit (platform)
26.06.2026	6863b8a48ceb0532dce67a0fbbd48b3ea7e08286	Commit for strategy-template re-review (platform)
30.06.2026	c28061e4f3767826337f9b72d03f0f5641a233b5	Commit for re-audit (lz-v2)
07.07.2026	30f27218e890a5c69b53b93d64c5fce52fbe8294	StrategyTemplate re-audit fixes (platform)
07.07.2026	a6ed2b5c1418076325bd6193971f7f7eea16797c	LayerZero ordered-delivery recovery fix (lz-v2)

Mainnet Deployments

At this stage, the contracts have not yet been deployed. Verification that the deployed contracts correspond to the audited code will be conducted once the client provides the addresses of the deployed contracts.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

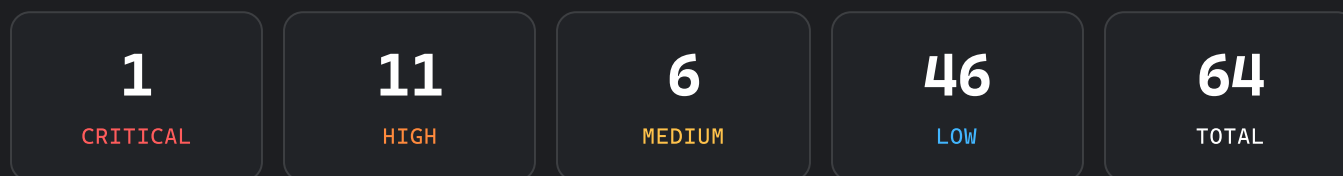
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

ID	FINDING	SEVERITY	STATUS
C-1	Reshuffling Gateway Assets Can Be Drained via Share Transfer	● CRITICAL	● FIXED
H-1	Strategy and Vault Can Be Bricked due to State Desync in Emergency Exit	● HIGH	● FIXED
H-2	Unauthorized Claim on Behalf of Users Leads to Fund Loss in Repairing Mode	● HIGH	● FIXED
H-3	Vault DoS When Container Is Added during Active Batch	● HIGH	● FIXED
H-4	Users Lose Rights to Future Recovered Assets After First Claim in Repairing Mode	● HIGH	● FIXED
H-5	Inconsistent Claim Logic between claimDeposit() and claimReshufflingGateway() Leads to Fund Loss	● HIGH	● FIXED
H-6	Missing Binding between Burn and Receiver Messages Can Lock or Miscredit Funds	● HIGH	● FIXED
H-7	Buffered Deposits and Pending Withdrawals Locked in Repairing Mode	● HIGH	● FIXED
H-8	Inconsistent Bridge Return Amount Breaks Cross-Chain Claims	● HIGH	● FIXED
H-9	Withdrawal Batch BPS Truncation Causes Up to ~50% Loss for Withdrawers Near Basis-Point Boundaries	● HIGH	● FIXED

ID	FINDING	SEVERITY	STATUS
H-10	Unclaimed Withdrawal Shares Corrupt totalSupply-Based Calculations	● HIGH	● FIXED
H-11	Share Price Inflation Attack	● HIGH	● ACKNOWLEDGED
M-1	Missing notInRepairingMode Modifier in withdraw() Function	● MEDIUM	● FIXED
M-2	Unsafe Token Approval Across Multiple Contracts	● MEDIUM	● FIXED
M-3	Missing handleReceiveUnfinalizedMessage() Callback Can Cause Claim Reverts	● MEDIUM	● FIXED
M-4	Withdrawal Batch Processing Can Be Indefinitely Stuck Due to minWithdrawBatchRatio Threshold	● MEDIUM	● FIXED
M-5	depositWithPermit() Is Front-Runnable	● MEDIUM	● FIXED
M-6	lastResolvedDepositBatchBlock and lastResolvedWithdrawBatchBlock Are Never Updated After Initialization	● MEDIUM	● FIXED
L-1	CustomPool.swap() Transfers Tokens from receiver instead of msg.sender	● LOW	● FIXED
L-2	Fee Cap Validation Uses Strict Inequality (<) Instead of Less Than or Equal (<=)	● LOW	● FIXED
L-3	Insufficient Cache Size for Bridge Retries	● LOW	● FIXED
L-4	UniversalAdapter Transfers Full Balance Instead of Swap Delta	● LOW	● FIXED
L-5	Ambiguous Cache Key for Identical Bridge Transactions	● LOW	● FIXED
L-6	Deprecated Chainlink latestAnswer Function	● LOW	● FIXED
L-7	Unused Error Definitions	● LOW	● FIXED
L-8	Missing Support for LayerZero Token Fee Payments in MessageRouter	● LOW	● FIXED

ID	FINDING	SEVERITY	STATUS
L-9	Incorrect Strict Inequality in Slippage Check Prevents Exact Match Success	● LOW	● FIXED
L-10	Redundant Caching of Received Messages Reduces Retry Window	● LOW	● FIXED
L-11	Protocol DoS via skipDepositBatch in Empty Vault	● LOW	● FIXED
L-12	ShiftOApp Refund Address Uses tx.origin	● LOW	● FIXED
L-13	RingCacheLibrary Defines Unused DEFAULT_CACHE_SIZE Constant	● LOW	● FIXED
L-14	Anyone Can Grief Container Deposit Reporting by Transferring Whitelisted Tokens	● LOW	● ACKNOWLEDGED
L-15	Strategy Exit and Enter Uses Pull Pattern for Transferring Tokens, Reducing Composability	● LOW	● ACKNOWLEDGED
L-16	CCTPv2 blacklistDomain Does Not Validate domainId Matches Stored Value and Leaks Stale Reverse Mappings	● LOW	● FIXED
L-17	AcrossBridgeAdapter Validates fillDeadline But Never Uses It in the Bridge Call	● LOW	● FIXED
L-18	UniswapV3Adapter whitelistPath Allows Zero-Hop Paths	● LOW	● FIXED
L-19	Naming and Error Message Clarity	● LOW	● ACKNOWLEDGED
L-20	Redundant Unchecked Loop Increments	● LOW	● FIXED
L-21	Vault claim* Reverts When Nothing to Claim, Breaking Integrations	● LOW	● FIXED
L-22	Meaningless Balance-Delta Check After Bridging	● LOW	● FIXED
L-23	Too Strict Slippage Check on bridgedAmount	● LOW	● FIXED
L-24	Nonce Check Can Break Delivery on Message Reordering	● LOW	● FIXED
L-25	No Way to Remove Predefined Swaps	● LOW	● FIXED

ID	FINDING	SEVERITY	STATUS
L-26	Strict Inequalities in Deposit Limits	● LOW	● FIXED
L-27	setContainerWeights() Allows Duplicate Container Addresses	● LOW	● FIXED
L-28	startDepositBatchProcessing Can Revert on Zero Container Amount	● LOW	● FIXED
L-29	Single EMERGENCY_PAUSER_ROLE Controls Both pause() and unpause() With No Threshold Differentiation	● LOW	● ACKNOWLEDGED
L-30	Missing setConfig() Wrapper for Library Parameter Tuning	● LOW	● FIXED
L-31	Missing _disableInitializers()	● LOW	● FIXED
L-32	blacklistDomain Leaves Stale domainId in Domain Storage	● LOW	● FIXED
L-33	CustomOracleWrapper.getPrice() — Returns Zero for Unset Tokens Without Reverting	● LOW	● FIXED
L-34	ChainlinkOracleWrapper — Single Global Staleness Threshold for All Feeds	● LOW	● FIXED
L-35	ChainlinkOracleWrapper — No L2 Sequencer Uptime Check	● LOW	● ACKNOWLEDGED
L-36	PriceOracleAggregator — Single Oracle per Token With No Fallback	● LOW	● ACKNOWLEDGED
L-37	Container, CrossChainContainer, StrategyTemplate — Missing Storage Gaps (__gap) in Base Abstract Contracts	● LOW	● FIXED
L-38	CrossChainContainer.setBridgeAdapter() and ReshufflingGateway.blacklistBridgeAdapter() — De-Listed Adapters Retain Token Approvals	● LOW	● ACKNOWLEDGED
L-39	StrategyContainer.isStrategyNavUnresolved() — Incorrect Result for Non-Existent Strategies	● LOW	● FIXED
L-40	Vault.claimWithdraw() — Shares Permanently Locked When Notion Rounds to Zero	● LOW	● FIXED

ID	FINDING	SEVERITY	STATUS
L-41	Empty DepositRequest Leaves ContainerAgent in Unrecoverable DepositRequestReceived	● LOW	● FIXED
L-42	Strict Slippage Inequality in _enterToState() Reverts on Exact minNavDelta Match	● LOW	● FIXED
L-43	defaultPriceFeedStalenessThreshold Has No Post-Construction Setter in ChainlinkOracleWrapper	● LOW	● FIXED
L-44	StrategyTemplate.emergencyExit() — Failure Handling Is Unreachable Unless emergencyExitMaxSlippage == MAX_BPS	● LOW	● FIXED
L-45	StrategyTemplate — Dangling Router Allowance After a No-Route Swap	● LOW	● FIXED
L-46	ShiftOApp Ordered Delivery Omits the skipInboundNonce Recovery Helper	● LOW	● FIXED

2.1 Critical

C-1 Reshuffling Gateway Assets Can Be Drained via Share Transfer

● CRITICAL

● FIXED AS OF 8392c1b3

DESCRIPTION

The `RepairingMode` exit mechanism allows users to drain the `ReshufflingGateway` of recovered assets by repeatedly transferring their vault shares to new addresses and claiming.

In `Vault.claimReshufflingGateway()`, the contract checks if an address has already claimed using a simple mapping:

```
require(!_hasClaimedReshufflingGateway[account], AlreadyClaimed());
_hasClaimedReshufflingGateway[account] = true;
IEReshufflingGateway(reshufflingGateway).withdraw(account);
```

The `ReshufflingGateway.withdraw()` function calculates the amount of tokens to send based on the user's current vault share balance:

```
uint256 shares = IERC20(vault).balanceOf(account);
uint256 totalShares = IERC20(vault).totalSupply();
// ...
uint256 amount = balance.mulDiv(shares, totalShares);
```

Crucially, vault shares are never burned or locked during this process. An attacker can:

1. Call `claimReshufflingGateway()` to receive their pro-rata share of the gateway's current assets.
2. Transfer their vault shares to a fresh address they control.
3. Call `claimReshufflingGateway()` from the new address to receive the pro-rata share again.
4. Repeat this process until the `ReshufflingGateway` is drained.

This issue is classified as **Critical** severity because it enables complete theft of assets held in the `ReshufflingGateway` with no possibility of recovery.

RECOMMENDATION

We recommend burning the user's vault shares within the `Vault.claimReshufflingGateway()` function after calling the gateway.

2.2 High

H-1 Strategy and Vault Can Be Bricked due to State Desync in Emergency Exit

● HIGH

● FIXED AS OF 8392c1b3

DESCRIPTION

A logic flaw in the interaction between `StrategyTemplate` and `StrategyContainer` during emergency situations can lead to a strategy being permanently locked in NAV resolution mode, rendering it unusable and its funds unreachable via normal operations.

The issue resides in the `StrategyTemplate.emergencyExit()` function:

```
vars.isResolvingEmergency =
    IStrategyContainer(_strategyContainer).isResolvingEmergency();
...
if (!vars.isResolvingEmergency) {
    IStrategyContainer(_strategyContainer).startEmergencyResolution();
}
if (!_navResolutionMode) {
    _setNavResolutionMode(true);
}
```

The function only calls `startEmergencyResolution()` on the container if the container is not already in emergency mode. However, the container's `startEmergencyResolution()` is responsible for setting the specific strategy's bit in the `_strategyUnresolvedNavBitmask`.

Example Scenario:

1. Strategy A enters emergency mode. The Container sets `_isResolvingEmergency = true` and sets the bit for Strategy A in its `_strategyUnresolvedNavBitmask`.
2. Strategy B subsequently calls `emergencyExit()`.
3. Strategy B observes that `isResolvingEmergency()` is already `true` (due to Strategy A), so it skips the call to `container.startEmergencyResolution()`.
4. Strategy B sets its internal flag `_navResolutionMode = true`.
5. The Container now has `_isResolvingEmergency = true` but its `_strategyUnresolvedNavBitmask` only contains the bit for Strategy A.
6. The Emergency Manager resolves Strategy A. The `_strategyUnresolvedNavBitmask` in the container becomes 0.
7. The Emergency Manager calls `completeEmergencyResolution()`. Since the `_strategyUnresolvedNavBitmask` is 0, the function succeeds and sets `_isResolvingEmergency = false`.
8. The Container is back in normal mode, but Strategy B is still stuck with `_navResolutionMode = true`.

Any attempt to fix Strategy B via `acceptNav()` will fail. `acceptNav()` calls `container.resolveStrategyNav()`, which reverts if the strategy's bit is not set in the container's bitmask. Since the bit for Strategy B was never set, it can never be cleared, and `_navResolutionMode` will remain `true`, blocking `harvest()`, `enter()` and `exit()` operations.

The impact is not limited to the container and can escalate to a permanent deadlock of the entire Vault system. When a deposit batch starts, the Vault's status becomes `DepositBatchProcessingStarted`. It can only return to `Idle` via `resolveDepositBatch()`, which requires `isDepositReportComplete()`, meaning every registered container must call `reportDeposit()`. The bricked strategy prevents the container from ever reaching the `AllStrategiesEntered` status required to call `vault.reportDeposit()`.

There is no escape mechanism:

- The Vault has no "emergency reset" function for a batch in progress.
- `removeContainer` or `setContainerWeights` (to remove the dead container) both require the Vault to be in `Idle` status.
- The Vault cannot reach `Idle` because the container is stuck.
- The container cannot be reset or have its strategy removed because it is not in `Idle` status.

This issue is classified as **High** severity because in the worst case the entire batch and Vault are deadlocked and protocol funds are blocked until admin recovery via contract upgrade, while in the best case only a single strategy is bricked and its funds remain locked.

RECOMMENDATION

We recommend removing the conditional check in `StrategyTemplate.emergencyExit()` and always calling `startEmergencyResolution()` to ensure every strategy that enters emergency mode is correctly registered in the container's bitmask.

```
// StrategyTemplate.sol

// Remove: if (!vars.isResolvingEmergency) {
IStrategyContainer(_strategyContainer).startEmergencyResolution();
// }
```

H-2 Unauthorized Claim on Behalf of Users Leads to Fund Loss in Repairing Mode

● HIGH

● FIXED AS OF 8392c1b3

DESCRIPTION

The `Vault.claimReshufflingGateway()` function allows any caller to trigger a claim on behalf of any other user. When the manager activates repairing mode, an attacker can immediately monitor the transaction and, before recovered funds are fully transferred to the `ReshufflingGateway`, call `claimReshufflingGateway(user_address)` for each shareholder. Since the gateway is empty or only partially funded at that moment, users receive 0 or a fraction of their funds, but their `_hasClaimedReshufflingGateway` flag is set to `true`, permanently blocking them from future recoveries.

```
function claimReshufflingGateway(address account) external nonReentrant {
    require(isRepairing, NotInRepairingMode());
    require(!_hasClaimedReshufflingGateway[account], AlreadyClaimed());

    _hasClaimedReshufflingGateway[account] = true;

    IReshufflingGateway(reshufflingGateway).withdraw(account);
    emit ReshufflingGatewayClaimed(account);
}
```

This issue is classified as **High** severity because it can potentially block up to 100% of user funds from being claimed through the normal flow and can be triggered by any actor. The blocked funds remain on the gateway contract and they can be recovered only by upgrading the gateway contract and then distributing them to affected users via a separate claim or distribution mechanism.

RECOMMENDATION

We recommend restricting the `claimReshufflingGateway()` function so that users can only claim for themselves, and ensuring that the call reverts if no funds were actually transferred.

H-3 Vault DoS When Container Is Added during Active Batch

● HIGH

● FIXED AS OF 8392c1b3

DESCRIPTION

The `Vault.addContainer()` function allows the `CONTAINER_MANAGER_ROLE` to add a new container to the protocol at any time, regardless of the current vault status. If a container is added while a deposit or withdrawal batch is in progress (i.e., status is not `Idle`), it leads to a denial of service for the vault.

The vault tracks batch completion using a bitmask-based approach in the `isDepositReportComplete()` and `isWithdrawReportComplete()` functions:

```
function isDepositReportComplete() public view returns (bool) {
    return _depositReportBitmask == (1 << _containers.length()) - 1;
}

function isWithdrawReportComplete() public view returns (bool) {
    return _withdrawReportBitmask == (1 << _containers.length()) - 1;
}
```

Example Scenario:

1. If a container is added when `status != Idle`, the `_containers.length()` increases immediately.
2. The completion check functions now expect an additional bit in the bitmask corresponding to the new container.
3. However, the newly added container was not part of the active batch when the batch started (e.g., in `startDepositBatchProcessing` or `startWithdrawBatchProcessing`), so it will never be asked to report and thus will never update the bitmask.
4. The `resolveDepositBatch()` or `resolveWithdrawBatch()` functions will revert indefinitely because the report will never be "complete".
5. There is no way to reset the status to `Idle` once the vault is stuck:
 - `skipDepositBatch()` requires `status == Idle`.
 - `skipWithdrawBatch()` requires `status == DepositBatchProcessingFinished`.
 - `resolveDepositBatch()` and `resolveWithdrawBatch()` are the only functions that transition the state forward, but they are both blocked by the bitmask check.
6. To fix this, an administrator might try to remove the new container or change its weight using `setContainerWeights()`. However, `setContainerWeights()` also requires `status == VaultStatus.Idle`, which is impossible to reach since the current batch cannot be resolved.

This issue is classified as **High** severity because the vault is put under permanent DoS, funds on the contract are fully blocked from withdrawal through normal flow, and the only way for an admin to recover them is to upgrade the affected contracts.

RECOMMENDATION

We recommend adding a status check to the `addContainer()` function to ensure it can only be called when the vault is in the `Idle` state.

H-4 Users Lose Rights to Future Recovered Assets After First Claim in Repairing Mode

● HIGH

● FIXED AS OF 8392c1b3

DESCRIPTION

The `RepairingMode` claiming mechanism prevents users from claiming their share of assets that arrive in the `ReshufflingGateway` after their first claim, leading to loss of funds.

The `Vault.claimReshufflingGateway()` function uses a one-time boolean flag per account:

```
require(!_hasClaimedReshufflingGateway[account], AlreadyClaimed());
_hasClaimedReshufflingGateway[account] = true;
```

The admin first enables repairing mode and after that recovered funds start arriving at the `ReshufflingGateway`. When a user sees that they can claim their share, they may claim once (e.g., after the first batch has arrived) and assume they will be able to claim again when more funds arrive later. However, the flag is set to `true` on the first claim, so any later call to `claimReshufflingGateway()` will revert. That user thus loses the right to their share of funds that arrive after their first claim; those funds remain stuck for them.

This issue is classified as **High** severity because it causes a lock of user funds until admin recovery via contract upgrade, and recovery requires the admin to set up a complex token distribution mechanism for affected users.

RECOMMENDATION

We recommend reconsidering the repairing-mode flow at an architectural level, as it currently exhibits multiple design issues. For this specific finding, we recommend considering an approach where recovered funds first arrive at the gateway, then the admin explicitly approves or confirms that all expected funds have arrived, and only after that confirmation are users allowed to claim via `claimReshufflingGateway()`. That way users claim once against the full recovered amount and do not lose access to funds that would otherwise arrive later.

H-5 Inconsistent Claim Logic between `claimDeposit()` and `claimReshufflingGateway()` Leads to Fund Loss

● HIGH

● FIXED AS OF 8392c1b3

DESCRIPTION

The interaction between `claimDeposit()` and `claimReshufflingGateway()` in `RepairingMode` creates an inconsistency that can lead to loss of user funds and locked assets in the `ReshufflingGateway`.

When a user has unclaimed deposits (`pendingBatchDeposits[batchId][user] > 0`) but calls `claimReshufflingGateway()` before `claimDeposit()`:

- `ReshufflingGateway.withdraw()` calculates the user's share based on their current `balanceOf(user)` only.
- This balance can be lower than the user's effective share ownership, because additional shares are still pending inside the Vault (not yet transferred to the user) until `claimDeposit()` is called.
- As a result, the user claims from the gateway as if they own only the currently-held shares, while they may later receive additional shares from unclaimed deposits.
- The flag `_hasClaimedReshufflingGateway[user] = true` is set, preventing future gateway claims.
- The user can then call `claimDeposit()` to receive the remaining shares, but they cannot claim from the gateway again to receive the portion of gateway funds corresponding to these newly claimed shares.

Example Scenario:

1. User already has 100 shares on their wallet from previously claimed batches.
2. User also has an additional deposit that would mint 200 shares: either the shares are already on the Vault from a resolved batch and the user has not called `claimDeposit()` yet, or (more realistically) the batch is not yet processed and the user will be able to claim those shares once it is resolved.
3. User calls `claimReshufflingGateway()` first and gateway payout is calculated only for the 100 wallet shares, and `_hasClaimedReshufflingGateway[user]` is set to `true`.
4. User then calls `claimDeposit()` and receives the additional 200 shares on their wallet.
5. User cannot claim from the gateway again for the 200 shares, resulting in loss of the corresponding portion of gateway funds.

This issue is classified as **High** severity because it leads to loss of user funds, and the problem can only be resolved potentially by upgrading the contracts.

RECOMMENDATION

We recommend reconsidering the `RepairingMode` flow at an architectural level, as it currently exhibits multiple design shortcomings. There is no trivial fix for this ordering issue because the final share balance is only known after the deposit batch is resolved and `Vault.claimDeposit()` is executed, while in `RepairingMode` a batch may never become claimable.

When addressing this issue, the team may consider one or a combination of the following approaches:

- Enforcing ordering only when possible: block `Vault.claimReshufflingGateway()` only while the user has unclaimed deposits in resolved batches, forcing `Vault.claimDeposit()` first.
- Making the gateway claim incremental: allow multiple `Vault.claimReshufflingGateway()` calls

and pay only the delta attributable to newly minted shares, removing ordering dependence.

- Handling unresolved/stuck batches: do not block gateway claims due to deposits that are not yet claimable. In such cases, users should be able to claim based on their current share balance (and, if the batch later becomes claimable and mints additional shares, be able to claim the incremental amount under the chosen approach).

H-6 Missing Binding between Burn and Receiver Messages Can Lock or Miscredit Funds

● HIGH

● FIXED AS OF 85d41f98

DESCRIPTION

`CCTPv2BridgeAdapter.claimCCTPBridge()` processes two independent CCTP messages by calling `MessageTransmitterV2.receiveMessage()` twice: first for the burn/mint message (`bridgeMessage`), then for the receiver/app message (`messageMessage`). The adapter computes `amountClaimed` solely as the change in its USDC balance after the first `MessageTransmitterV2.receiveMessage()` call and then uses transient storage to pass that value to the handler executed during the second `receiveMessage()` call.

However, `CCTPv2BridgeAdapter.claimCCTPBridge()` does not validate that `bridgeMessage` and `messageMessage` belong to the same original bridge operation (e.g., no shared nonce, hash commitment, or other linkage). There are two different scenarios which can be exploited due to the issue:

1. An attacker can submit a small burn/mint message (with `mintRecipient` = destination adapter) and a victim's receiver/app message in a single `claimCCTPBridge()` call. This consumes the victim receiver message nonce but only credits the victim with the attacker-controlled small `amountClaimed` . The victim's real burn/mint message can then no longer be paired with its receiver message, leaving the victim's bridged USDC locked.
2. When two containers each send a bridge (e.g. 10,000 USDC to receiver A, 50,000 USDC to receiver B), an attacker can call `claimCCTPBridge()` with one container's burn message and the other container's receiver message. One receiver is credited with the wrong amount (e.g. A gets 50,000 instead of 10,000, B gets nothing), the other container's receiver message may be consumed with the wrong amount or the other container's burn may remain unclaimable, and funds can end up stuck or miscredited.

This issue is classified as **High** severity, because it can lead to user funds being locked or miscredited and can be triggered by any actor able to obtain attestations and call

`CCTPv2BridgeAdapter.claimCCTPBridge()` . Affected funds remain locked, the only way to resolve the situation is for the admin to upgrade the contracts.

RECOMMENDATION

We recommend either introducing a binding between the burn/mint message and the receiver message and validating this binding on the destination when handling received messages, or restricting `claimCCTPBridge()` to a dedicated role (e.g. relayer or bridge-claim role) so that only trusted callers can submit claims and mix-and-match is not permissionless.

H-7 Buffered Deposits and Pending Withdrawals Locked in Repairing Mode

● HIGH

● FIXED AS OF 8392c1b3

DESCRIPTION

In `RepairingMode` there is no way to exit once activated. Two flows can trap user funds in the `Vault` with no in-protocol recovery and no inclusion in `ReshufflingGateway` pro-rata distribution (which is based on share balance).

Example 1 — Buffered deposits:

- 1.1. User deposits notion via `Vault.deposit()` before a batch is started. Notion is in `Vault`, recorded in `bufferedDeposits` and `pendingBatchDeposits`.
- 1.2. Emergency Manager activates `RepairingMode` before `startDepositBatchProcessing()` is called.
- 1.3. `startDepositBatchProcessing()` has `notInRepairingMode`, so the batch never starts. User never gets shares and cannot call `claimDeposit()` (batch never resolved). No refund function exists.
- 1.4. Notion stays locked in `Vault` indefinitely. User gets nothing from `ReshufflingGateway` because payout is by share balance and they have no shares.

Example 2 — Pending withdrawals:

- 2.1. User calls `Vault.withdraw()`: shares move from user to `Vault` and are recorded in `pendingBatchWithdrawals`. User wallet share balance drops.
- 2.2. Emergency Manager activates `RepairingMode`. Standard withdraw batch may never resolve. Emergency exit is via `ReshufflingGateway.withdraw(account)`, which uses `balanceOf(account)`.
- 2.3. User's shares are in the `Vault`, not in their wallet, so `balanceOf(user)` is zero. Gateway pays them nothing. There is no `cancelWithdraw()` to return shares.
- 2.4. User is blocked from both the standard batch resolution and the emergency gateway exit. Funds are stuck in the `Vault`.

This issue is classified as **High** severity: in either flow, user funds (notion tokens or shares) can be blocked with no in-protocol way to recover and no participation in the emergency gateway distribution. Until the vault is upgraded by the admin, the blocked tokens or shares remain locked.

RECOMMENDATION

We recommend reconsidering the repairing-mode flow at an architectural level, as it currently exhibits multiple design issues. For this specific finding, the following functionality could be implemented:

- For buffered deposits: add an `emergencyRefundDeposit()` so notion recorded in `pendingBatchDeposits` can be recovered by users in `RepairingMode` when the batch will not be processed.
- For pending withdrawals: add `cancelWithdraw()` in `RepairingMode` so users can cancel their pending withdrawal request and receive back their shares from the `Vault` (the shares currently recorded in `pendingBatchWithdrawals` and not processable in the batch); that way users regain a non-zero share balance and can claim their pro-rata share from `ReshufflingGateway`.

H-8 Inconsistent Bridge Return Amount Breaks Cross-Chain Claims

● HIGH

● FIXED AS OF 85d41f98

DESCRIPTION

`CrossChainContainer._bridgeToken()` treats the return value of `IBridgeAdapter(bridgeAdapter).bridge(...)` as the amount that will be received on the destination chain and later claimed. This value is sent cross-chain and tracked as an expected amount. On the destination chain, the claim path enforces that the actual claimed amount is at least the expected amount:

```
uint256 expectedAmount = _expectedTokenAmounts[token];
// ...
IBridgeAdapter(bridgeAdapter).claim(token);
uint256 amountReceived = IERC20(token).balanceOf(address(this)) - balanceBefore;
require(amountReceived >= expectedAmount,
        InsufficientBridgeAmount(expectedAmount, amountReceived));
```

However, bridge adapters do not consistently return the same semantic meaning (gross vs net) for `bridge()`:

- `AcrossBridgeAdapter` returns `minAmount = instruction.amount - acrossPayload.fee` (net amount expected to be received).
- `CCTPv2BridgeAdapter` returns `instruction.amount` (gross amount), while Circle CCTP Fast Transfers charge a fee, so the destination receives `amount - fee` (net). Documentation: <https://developers.circle.com/cctp/concepts/fees>.

This issue is classified as **High** severity because if fees are charged on the transfer and `CCTPv2BridgeAdapter` returns the gross amount, the destination chain will expect more than it can actually receive/claim, and `claim()` will revert. The container then cannot clear `claimCounter` and cannot progress to the `BridgeClaimed` / report stage. As a result, the container remains stuck: it never calls `reportDeposit()` or `reportWithdraw()` to the Vault, so the deposit or withdrawal batch that depends on this container never completes. Users' bridged funds remain in flight (stuck on the destination adapter or in an incomplete batch). Cross-chain deposit/withdraw processing is stalled. The only way to resolve this is to upgrade the contracts.

RECOMMENDATION

We recommend standardizing `IBridgeAdapter.bridge()` return semantics to always return a conservative net/claimable amount (i.e., a value guaranteed to be `<=` the amount that will be claimable on the destination chain).

H-9 Withdrawal Batch BPS Truncation Causes Up to ~50% Loss for Withdrawers Near Basis-Point Boundaries

● HIGH

● FIXED AS OF 8392c1b3

DESCRIPTION

`_calculateSharesPercent()` computes `shares.mulDiv(MAX_BPS, totalSupply())` which truncates to an integer number of basis points (0.01% granularity). This truncated value is passed to every container's `registerWithdrawRequest()`, which passes it to every strategy's `exit(share)`. The example of compound rounding chain:

1. **Vault:** `batchSharesPercent` truncated from, e.g., 1.999 BPS to 1 BPS (50% precision loss).
2. **Strategy (protocol exit):** `protocolBalance * share / BPS` rounds DOWN per strategy.
3. **Strategy (idle portion):** `idleBalance * share / BPS` rounds DOWN per strategy.

The worst case occurs when the exact BPS is just below an integer boundary. Example at 10M TVL: a withdrawer with 1,999 shares (exact 1.999 BPS) gets truncated to 1 BPS. Strategies exit 0.01% instead of 0.01999%. The withdrawer receives ~1,000 tokens instead of ~1,999 tokens — a ~50% loss. The lost value remains in the strategies, benefiting remaining shareholders.

Maximum loss per batch scales linearly with TVL: `totalSupply / MAX_BPS`. At 10M TVL, this is 1K tokens per batch, or up to 365K tokens/year with daily batches (3.65% of TVL).

RECOMMENDATION

We recommend using higher precision for the share calculation (e.g., 1e18 instead of 10,000 BPS). Propagate the full-precision share fraction to containers and strategies rather than truncating to an integer BPS. Alternatively, compute the exact token amounts to withdraw at the vault level and pass absolute amounts rather than percentages.

H-10 Unclaimed Withdrawal Shares Corrupt totalSupply-Based Calculations

● HIGH

● FIXED AS OF 79ca6dc8

DESCRIPTION

When a withdrawal batch is resolved via `Vault.resolveWithdrawBatch()`, shares are NOT burned. They remain in `totalSupply()` as shares held by the Vault until users call `Vault.claimWithdraw()`, which triggers `_burn()`. These "zombie shares" have no NAV backing — their proportional strategy NAV was already exited — but they inflate `totalSupply()`.

In `Vault.resolveDepositBatch()`, the formula:

```
vars.batchShares = vars.batchDeltaNav.mulDiv(vars.totalSupplyCached, vars.totalNav0);
```

uses the inflated `totalSupplyCached`, minting more shares than warranted. Similarly, `Vault._calculateSharesPercent()` computes a smaller exit fraction for the next withdrawal batch because of the inflated denominator.

Example scenario.

`totalSupply = 10_000`, NAV = 10_000 (1:1). 5 shareholders each hold 1_000 shares and submit `withdraw(100%)`. Their 5_000 shares transfer to the Vault. After `resolveWithdrawBatch()` those shares are NOT burned — `totalSupply` remains 10_000, but strategy NAV drops to 5_000 (half the positions were exited).

A new depositor now deposits 5_000 notion. Containers deploy it: `nav0 = 5_000`, `nav1 = 10_000`, `batchDeltaNav = 5_000`.

```
batchShares = 5_000 × 10_000 / 5_000 = 10_000 // minted
fair value: 5_000 × 5_000 / 5_000 = 5_000
```

The depositor receives 10,000 shares instead of the correct 5,000 — a 2× over-allocation. After `claimWithdraw()` burns the 5,000 zombie shares, the remaining 5 shareholders hold 5,000 shares (33.3% of NAV) while the new depositor holds 10,000 shares (66.7%).

The new depositor controls 66.7% of a 10_000 NAV vault despite contributing only 50% of the assets. Remaining shareholders lose **16.7 percentage points** of NAV ownership — a direct, quantifiable loss transferable to any depositor who times their deposit between `resolveWithdrawBatch()` and the subsequent `claimWithdraw()` calls.

This issue is classified as High severity, because a depositor can systematically extract value from existing shareholders in proportion to outstanding zombie shares and withdrawal batch size, with a direct, quantifiable redistribution of NAV ownership.

RECOMMENDATION

We recommend introducing a `pendingBurnShares` state variable that tracks shares pending burn and subtracting it from `totalSupply()` wherever share-based calculations occur:

```
// In resolveWithdrawBatch():
pendingBurnShares += withdrawBatchTotalShares[previousBatchId];
```

```
// In resolveDepositBatch() and _calculateSharesPercent():
uint256 effectiveSupply = totalSupply() - pendingBurnShares;

// In claimWithdraw() (unchanged burn, just decrement counter):
pendingBurnShares -= vars.withdrawnShares;
_burn(address(this), vars.withdrawnShares);
```

H-11 Share Price Inflation Attack

● HIGH

● ACKNOWLEDGED

DESCRIPTION

The **Vault** is vulnerable to a share price inflation attack. The issue arises if an attacker becomes the sole depositor in the first batch by filling `maxDepositBatchSize` with their own funds. As the only initial shareholder, the attacker can withdraw almost all shares (leaving only 1 wei), then transfer assets directly to the strategy without minting new shares. This drastically reduces the share supply while increasing NAV, artificially inflating the share price. Consequently, subsequent depositors receive far fewer shares than expected, enabling the attacker to extract value via their remaining dust share position.

Example Scenario:

1. Let `maxDepositBatchSize` be 10,000 tokens. The attacker deposits 10,000 tokens, possibly in multiple sequential deposits, to fill the first batch exactly to `maxDepositBatchSize` and become the sole depositor.
2. The attacker receives 10,000 shares at the initial 1:1 ratio.
3. The attacker withdraws almost all shares using two consecutive withdrawals with carefully chosen percentages, leaving only 1 wei of shares in circulation.
4. The attacker transfers 10,001 tokens directly to the strategy contract, bypassing the normal deposit mechanism.
5. This inflates the share price to an extreme level, with 10,001 tokens backing only 1 wei of shares.
6. A later legitimate user deposits 20,000 tokens but receives only about 1 share instead of the expected ~2.
7. The user loses roughly 5,000 tokens of value, which accrues to the attacker's remaining 1 wei share and can later be extracted.

This issue is classified as **High** severity because, in the worst case, it can lead to loss of funds for subsequent depositors. However, the likelihood of exploitation is constrained by the `maxDepositBatchSize` parameter and the timely detection of the attack.

RECOMMENDATION

We recommend using techniques such as dead shares, virtual shares, or an initial admin deposit to maintain a non-zero share supply and prevent share price inflation via supply reduction and direct donations.

CLIENT'S COMMENTARY

MixBytes: The development team acknowledges the issue and will make an initial admin deposit at the time of vault initialization to significantly reduce the attack surface. The admin deposit must be performed in the same transaction as the vault initialization to eliminate any front-running window between deployment and the protective deposit.

2.3 Medium

M-1 Missing `notInRepairingMode` Modifier in `withdraw()` Function

● MEDIUM

● FIXED AS OF 8392c1b3

DESCRIPTION

The `Vault.withdraw()` function lacks the `notInRepairingMode` modifier, which is present in `_deposit()`. In Repairing mode the protocol is in an emergency state and users are expected to exit only via claiming from the `ReshufflingGateway`. Allowing `withdraw()` in Repairing mode lets users submit withdrawal requests unaware of the emergency. Their shares are sent to the Vault (recorded in `pendingBatchWithdrawals`), and the withdraw batch may never complete (depending on vault status and operator actions). Users can end up with shares stuck in the vault and no gateway payout, or a reduced payout.

RECOMMENDATION

We recommend adding the `notInRepairingMode` modifier to the `withdraw()` function to prevent users from entering the withdrawal queue once the protocol has entered the emergency recovery state.

M-2 Unsafe Token Approval Across Multiple Contracts

● MEDIUM

● FIXED AS OF 8392c1b3, 59c80e13

DESCRIPTION

The codebase performs direct `approve()` calls in multiple locations instead of consistently using OpenZeppelin's `SafeERC20` helpers. As a result, interacting with non-standard ERC20 tokens (most notably USDT) can cause transactions to revert or behave unexpectedly. For example, some tokens do not return a boolean from `approve()`, and direct calls may revert when a return value is expected. Others require resetting a non-zero allowance to zero before setting a new value, otherwise `approve()` reverts. Some tokens may also return `false` instead of reverting, which can leave the contract in an inconsistent state.

The following locations in the codebase are affected:

1. `Vault.addContainer()` (line 254) and `Vault.setContainerWeights()` (line 277):

```
// addContainer
notion.approve(container, type(uint256).max);

// setContainerWeights
notion.approve(containers[i], 0);
```

2. `ContainerPrincipal.initialize()` (line 35):

```
IERC20(notion).approve(vault, type(uint256).max);
```

3. `ContainerLocal.initialize()` (line 27):

```
IERC20(notion).approve(vault, type(uint256).max);
```

4. `CrossChainContainer._approveTokenToBridgeAdapter()` (line 187):

```
IERC20(token).approve(bridgeAdapter, amount);
```

5. `AcrossBridgeAdapter._bridge()` (line 70, across-v3):

```
IERC20(instruction.token).approve(spookyPool, instruction.amount);
```

This issue is classified as **Medium** severity, because the protocol will use USDT, and in that case it can cause a contract lock that prevents critical operations.

RECOMMENDATION

We recommend using `forceApprove()` from OpenZeppelin's `SafeERC20` library instead of direct `approve()` calls across all affected locations.

M-3 Missing `handleReceiveUnfinalizedMessage()` Callback Can Cause Claim Reverts

● MEDIUM

● FIXED AS OF 85d41f98

DESCRIPTION

Circle CCTP v2 supports two finality modes: `Fast` and `Standard`. Per Circle [docs](#), only two finality thresholds are supported. Any `minFinalityThreshold` value below 1000 is treated as 1000, and any value above 1000 is treated as 2000.

When a message is attested with `finalityThresholdExecuted < 2000`, `MessageTransmitterV2.receiveMessage()` dispatches it to `handleReceiveUnfinalizedMessage()` on the message recipient, otherwise it dispatches to `handleReceiveFinalizedMessage()`. However, the adapter implements only `CCTPv2BridgeAdapter.handleReceiveFinalizedMessage()` and does not implement `CCTPv2BridgeAdapter.handleReceiveUnfinalizedMessage()`. As a result, when a message is attested as Fast, the claim reverts because that callback is missing.

RECOMMENDATION

We recommend either implementing `CCTPv2BridgeAdapter.handleReceiveUnfinalizedMessage()` or using only Standard transfers.

M-4 Withdrawal Batch Processing Can Be Indefinitely Stuck Due to `minWithdrawBatchRatio` Threshold

● MEDIUM

● FIXED AS OF 8392c1b3

DESCRIPTION

`startWithdrawBatchProcessing()` requires `batchSharesPercent >= minWithdrawBatchRatio`. If the total buffered withdrawal shares are below this threshold, the withdrawal batch cannot start. The `skipWithdrawBatch()` function can be used to skip, but this means those withdrawal requests are deferred to the next cycle. If the vault consistently has low withdrawal demand relative to TVL, withdrawals can be stuck indefinitely until either more users request withdrawals or the operator decreases `minWithdrawBatchRatio`.

With `minWithdrawBatchRatio = 1` (0.01%), at 10M TVL this means at least 1k tokens worth of shares must be queued. At 1B TVL, at least 100K tokens. Users with small positions may be unable to withdraw for extended periods.

RECOMMENDATION

We recommend increasing the ratio precision to 18 decimals and allowing withdrawals pending for more than N blocks, to be processed regardless of the ratio.

M-5 `depositWithPermit()` Is Front-Runnable

● MEDIUM

● FIXED AS OF 8392c1b3

DESCRIPTION

`Vault.depositWithPermit()` first calls `permit` and then performs the deposit. An attacker can front-run the user by submitting the same `permit` first, which consumes the nonce and makes the user's original transaction revert, causing a denial-of-service for deposit flows that rely on permits.

RECOMMENDATION

We recommend wrapping `permit` in a try/catch and proceeding with the deposit if the allowance is already sufficient, so a front-run does not break the flow.

M-6 `lastResolvedDepositBatchBlock` and `lastResolvedWithdrawBatchBlock` Are Never Updated After Initialization

● MEDIUM

● FIXED AS OF 79ca6dc8

DESCRIPTION

`Vault.sol` tracks two state variables intended to record the block number of the last resolved batch cycle:

```
uint256 public lastResolvedDepositBatchBlock;
uint256 public lastResolvedWithdrawBatchBlock;
```

Both are assigned once during initialization:

```
lastResolvedDepositBatchBlock = block.number;
lastResolvedWithdrawBatchBlock = block.number;
```

However, neither `resolveDepositBatch()` nor `resolveWithdrawBatch()` ever updates these variables. As a result, `batchProcessingDelay` computed inside `skipDepositBatch()` and `skipWithdrawBatch()` grows monotonically from the deployment block rather than from the last resolved batch:

```
uint256 batchProcessingDelay = block.number - lastResolvedDepositBatchBlock;
require(batchProcessingDelay < forcedBatchBlockLimit, CannotSkipForcedBatch());
```

Once `forcedBatchBlockLimit` blocks have elapsed since deployment, the check fails for any buffer at or above the forced threshold — regardless of how recently a batch was actually resolved. The timer is never reset.

The configurator (`CONFIGURATOR_ROLE`) can change `forcedBatchBlockLimit` via `setForcedBatchBlockLimit()`, so the problem is operationally solvable.

RECOMMENDATION

We recommend updating `lastResolvedDepositBatchBlock` at the end of `resolveDepositBatch()` and `lastResolvedWithdrawBatchBlock` at the end of `resolveWithdrawBatch()`.

2.4 Low

L-1 CustomPool.swap() Transfers Tokens from receiver instead of msg.sender

● LOW

● FIXED AS OF 3e47d624

DESCRIPTION

The `CustomPool.swap()` function transfers `tokenIn` from the `receiver` parameter instead of `msg.sender`. This is unsafe in general, because if any account has approved the `CustomPool` contract to spend its tokens, any third party can call `swap()` and cause the adapter to pull tokens from that account and send `tokenOut` to the same account.

Elsewhere in the protocol, swap adapters follow a single pattern: the caller is the one whose tokens are pulled.

In `CustomPool.swap()`, the code uses:

```
IERC20(tokenIn).safeTransferFrom(receiver, address(this), amountIn);
```

whereas other adapters (`OneInchAdapter`, `UniswapV3Adapter`, `UniversalAdapter`) use:

```
IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);
```

The intended flow is that routers (or other callers) approve the adapter and call `swap()`. The adapter should pull from `msg.sender`. Pulling from `receiver` instead breaks this pattern and, as above, allows approved-but-unintended token debits when users or contracts approve the pool.

RECOMMENDATION

We recommend changing `CustomPool.swap()` to transfer `tokenIn` from `msg.sender` instead of `receiver`.

L-2 Fee Cap Validation Uses Strict Inequality (<) Instead of Less Than or Equal (<=)

● LOW

● FIXED AS OF 59c80e13

DESCRIPTION

In the `AcrossBridgeAdapter` contract, the `_validatePayload()` function contains a check to ensure that the `acrossParams.fee` does not exceed the `feeCapPct`. The current implementation uses a strict less than (`<`) operator:

```
require(acrossParams.fee * MAX_FEE_CAP_BPS < feeCapPct * instruction.amount,
        FeeTooHigh(acrossParams.fee));
```

However, the `IAcrossV3Receiver` interface's documentation for `setFeeCapPct()` explicitly states that `feeCapPct` defines the maximum fee percentage allowed:

```
/**
 * @notice Sets the fee cap percentage
 * @dev Can only be called by addresses with the governance role.
 * The fee cap is used to validate
 * that bridge fees do not exceed the configured maximum percentage
 * of the bridged amount.
 */
```

If the calculated fee (`acrossParams.fee * MAX_FEE_CAP_BPS`) is exactly equal to the maximum allowed fee (`feeCapPct * instruction.amount`), the current `require` statement will cause the transaction to revert. This behavior is incorrect, as a fee that is precisely at the maximum allowed threshold should be considered valid and not "too high."

RECOMMENDATION

We recommend changing the fee cap check in `AcrossBridgeAdapter._validatePayload()` to use `<=` instead of `<`.

L-3 Insufficient Cache Size for Bridge Retries

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

The `_cache` in the `BridgeAdapter` contract, utilized for authorizing `retryBridge()` operations, is initialized as a `RingCache` with a maximum size of 8 elements. This small size creates a very narrow window to retry failed bridge transactions. After 8 unique bridge instructions have been processed, older cache entries are evicted from the ring buffer. Consequently, if `retryBridge()` is called for an instruction whose entry has been evicted, the transaction will revert with a `DoesNotExist` error, regardless of the actual bridge status. This prevents legitimate retries for transactions that have fallen out of the cache window.

RECOMMENDATION

We recommend making the cache size configurable at initialization, similar to `MessageRouter.initialize()` which accepts a `maxCacheSize` parameter, and ensuring `maxCacheSize` is not set too small so that the window for retries remains adequate.

L-4 UniversalAdapter Transfers Full Balance Instead of Swap Delta

● LOW

● FIXED AS OF 3e47d624

DESCRIPTION

In `UniversalAdapter.swap()`, the contract correctly computes the output delta (`amountOutAfter - amountOutBefore`) to validate slippage against `minAmountOut`, but then transfers the entire post-swap `tokenOut` balance (`amountOutAfter`) to the receiver instead of only the delta. Any pre-existing `tokenOut` balance on the adapter (e.g. from a direct transfer, dust, or mistaken send) is therefore sent to the receiver together with the swap output. Any caller can perform a swap with `receiver = self` and receive the full `tokenOut` balance of the adapter (swap output plus any pre-existing balance), so tokens previously left on the adapter can be drained to that receiver.

The protocol does not design any flow that leaves `tokenOut` on the adapter, so the only at-risk amounts are dust or mistaken transfers.

RECOMMENDATION

We recommend transferring only the swap output to the receiver, i.e. the delta (`amountOutAfter - amountOutBefore`).

L-5 Ambiguous Cache Key for Identical Bridge Transactions

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

In the `BridgeAdapter` contract, the cache key used for `retryBridge()` operations (derived from `instruction.token`, `instruction.chainTo`, `instruction.amount`, `receiver`) does not include any unique message or deposit identifier. If a container initiates two identical bridge transactions (same `token`, `chainTo`, `amount`, `receiver`), only one entry will be stored in the cache. This creates ambiguity: if one of these identical transactions fails and requires a retry, the `retryBridge()` function cannot differentiate which specific underlying deposit is being retried, potentially leading to confusion or unintended retries if the system expects unique authorization per message.

RECOMMENDATION

We recommend including a unique identifier in the cache key generation within `_cacheInstruction()` to ensure that each `bridge()` call creates a distinct, uniquely identifiable entry in the cache. This would allow `retryBridge()` to target specific failed transactions accurately, even if multiple identical transfers are initiated.

L-6 Deprecated Chainlink latestAnswer Function

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

The `ChainlinkOracleWrapper` contract uses the `latestAnswer()` function to fetch price data from `Chainlink` price feeds:

```
function getPrice() public view returns (uint256) {
    int256 price = IChainlinkPriceFeed(chainlinkFeed).latestAnswer();
    require(price > 0, Errors.IncorrectPrice(price));
    return uint256(price);
}
```

According to Chainlink documentation, the `latestAnswer()` function is deprecated. It does not return important metadata such as the timestamp of the update or the round ID, which are necessary to verify the freshness and validity of the price data. If the oracle stops updating or returns stale data, `latestAnswer()` will continue to return the last known value without any way for the caller to detect the issue.

RECOMMENDATION

We recommend replacing the call to `latestAnswer()` with `latestRoundData()` and implementing it with safety checks for price staleness.

L-7 Unused Error Definitions

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

The `Errors` library defines several error types that are never used anywhere in the audit scope. These unused definitions add unnecessary code bloat and can lead to confusion for developers who might expect these errors to be thrown in certain scenarios.

The following errors are defined but never referenced anywhere in the audit scope:

1. `OnlyAgent()` — line 8, never used
2. `AlreadyInitialized()` — line 13, never used (note: `RingCacheLibrary` defines its own `AlreadyInitialized(bytes32 id)` with a parameter)
3. `EmergencyModeEnabled()` — line 15, never used
4. `NotFound()` — line 32, never used (note: other errors with similar names such as `ContainerNotFound`, `StateNotFound` are used elsewhere, but `Errors.NotFound()` is not)
5. `InvalidDataLength()` — line 27, never used (note: `Codec.sol` defines and uses its own local `InvalidDataLength()` error)

RECOMMENDATION

We recommend removing the unused error definitions from the library to reduce code bloat and improve code clarity. If these errors were intended for future use, they should be documented as such, or if they represent removed functionality, they should be cleaned up.

L-8 Missing Support for LayerZero Token Fee Payments in MessageRouter

● LOW

● FIXED AS OF 98f9ff21

DESCRIPTION

`Shift0App` uses the LZ token fee path. In the adapter:

- `estimateFee(..., payInLz)` returns `messagingFee.lzTokenFee` when `payInLz` is true
- `send()` passes `MessagingFee({ nativeFee, lzTokenFee: zroFee })` into `_lzSend()`

However, the adapter cannot actually receive LZ token fees. In the router:

- `MessageRouter.send()` only forwards `msg.value` to the adapter and does not supply or approve LZ tokens

As a result, the LayerZero token cannot be used as a fee token in the current flow, and the LZ fee references in `Shift0App` are unusable without documentation or support in `MessageRouter`.

RECOMMENDATION

We recommend either clearing the code and documenting the limitation, or adding support for LZ token fees.

L-9 Incorrect Strict Inequality in Slippage Check Prevents Exact Match Success

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

The slippage check in the `StrategyTemplate._enter()` function uses a strict inequality (`>`) instead of a non-strict inequality (`>=`). This causes transactions to revert even when the resulting NAV increase exactly matches the provided `minNavDelta`.

```
require(
  vars.stateToNavAfterEnter > vars.stateToNavBeforeEnter + minNavDelta,
  SlippageCheckFailed(
    vars.stateToNavBeforeEnter,
    vars.stateToNavAfterEnter,
    minNavDelta
  )
);
```

In standard slippage protection logic, the minimum value (`minNavDelta`) should be inclusive: if the NAV increase is exactly equal to `minNavDelta`, the transaction should succeed. The current implementation forces the caller to supply a `minNavDelta` that is at least 1 unit smaller than the actual minimum requirement to avoid unnecessary reverts, which is counter-intuitive and can lead to precision-related issues.

RECOMMENDATION

We recommend replacing the strict inequality `>` with `>=` to allow transactions to succeed when the NAV increase exactly matches the `minNavDelta`.

L-10 Redundant Caching of Received Messages Reduces Retry Window

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

In `MessageRouter.receiveMessage()`, the contract caches every successfully received message in `_sendMessagesCache`:

```
function receiveMessage(
    bytes memory rawMessageWithPathAndNonce
)
    external
    nonReentrant
{
    // ... validation and delivery ...
    _cacheMessage(pathData.chainId, rawMessageWithPathAndNonce);
    // ...
}
```

However, `_sendMessagesCache` is primarily used to authorize `retryCachedMessage()`, which is a sender-side recovery mechanism for re-sending previously sent messages. Caching incoming messages is redundant and consumes cache capacity, accelerating ring-buffer eviction.

As a result, frequent incoming message traffic can evict outgoing messages that may still require a retry, causing `retryCachedMessage()` to revert with `RingCacheLibrary.DoesNotExists` even though the message should remain retryable from an operational perspective.

RECOMMENDATION

We recommend removing caching of messages inside `receiveMessage()` (or using a separate cache for received messages), keeping `_sendMessagesCache` dedicated to outgoing messages that may need `retryCachedMessage()`.

L-11 Protocol DoS via skipDepositBatch in Empty Vault

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

A logic flaw in the `Vault` state machine allows an operator to lock the protocol if `skipDepositBatch()` is called before any shares have been minted (i.e., when `totalSupply() == 0`).

The vulnerability follows this sequence:

1. The vault is in the `Idle` state with `totalSupply() == 0`
2. The operator calls `skipDepositBatch()`.
3. To return the vault to the `Idle` state (allowing new deposits), the operator must call either `startWithdrawBatchProcessing()` or `skipWithdrawBatch()`.
4. Both of these functions call the internal function `_calculateSharesPercent()`:

```
function _calculateSharesPercent(
    uint256 shares
)
    internal
    view
    returns (uint256)
{
    return shares.mulDiv(MAX_BPS, totalSupply());
}
```

Since `totalSupply()` is 0, this call will always revert due to a division by zero. As a result, the vault becomes stuck in the `DepositBatchProcessingFinished` state.

RECOMMENDATION

We recommend either forbidding `skipDepositBatch()` when the vault is empty (e.g. require `totalSupply() > 0` in `skipDepositBatch()`), or handling zero supply in `_calculateSharesPercent()` (e.g. return zero when `totalSupply() == 0`) so that `startWithdrawBatchProcessing()` and `skipWithdrawBatch()` can run and the vault can return to `Idle`.

L-12 ShiftOApp Refund Address Uses tx.origin

● LOW

● FIXED AS OF 98f9ff21

DESCRIPTION

In `ShiftOApp.send()`, the refund address passed to `_lzSend()` is `payable(tx.origin)`. LayerZero uses this address to refund overpaid native fees or to return value when a message fails on the source side.

In the current flow, the only callers that reach `MessageRouter.send()` (and thus the adapter's `send()`) are `ContainerPrincipal.sendDepositRequest`, `ContainerPrincipal.sendWithdrawRequest`, `ContainerAgent.reportDeposit`, and `ContainerAgent.reportWithdrawal`. All four are `onlyRole(OPERATOR_ROLE)` and forward `msg.value` from the operator. So in practice `tx.origin` is the operator who initiated the transaction, the operator pays the fees, and refunds go back to the operator—behavior is correct today.

Using `tx.origin` remains poor practice: it ties refund attribution to the transaction origin instead of an explicit contract guarantee. If the operator is a multisig, the refund goes to the EOA that initiated the transaction, not to the multisig. If new call paths are added (e.g. another role or a relayer), refunds could be misdirected. Replacing it with an explicit refund address (e.g. passed from the router or set via transient storage) would make the intended recipient clear and robust to future changes.

RECOMMENDATION

We recommend passing the refund address explicitly instead of using `tx.origin`. For example, extend the adapter interface so that `MessageRouter.send()` passes the intended refund recipient as a parameter into the adapter's `send()`, and have `ShiftOApp.send()` use that address in `_lzSend()`.

L-13 RingCacheLibrary Defines Unused DEFAULT_CACHE_SIZE Constant

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

In `RingCacheLibrary`, the constant `DEFAULT_CACHE_SIZE = 10` is defined but never used anywhere in the codebase. Callers of `RingCacheLibrary.initialize()` always pass an explicit cache size: `MessageRouter.initialize()` receives `maxCacheSize` as a parameter and passes it to `_sendMessagesCache.initialize(keccak256("SEND_CACHE"), maxCacheSize)`; `BridgeAdapter.__BridgeAdapter_init()` hardcodes the size in `_cache.initialize("BRIDGE_CACHE", 8)`.

RECOMMENDATION

We recommend either using `DEFAULT_CACHE_SIZE` where a default is appropriate (e.g. in `BridgeAdapter.__BridgeAdapter_init()` use `RingCacheLibrary.DEFAULT_CACHE_SIZE` instead of the hardcoded `8`, or add an overload of `initialize(RingCache, bytes32)` that calls `initialize(_cache, _id, DEFAULT_CACHE_SIZE)`, or removing the constant if no default is desired, to avoid dead code and clarify intent.

L-14 Anyone Can Grief Container Deposit Reporting by Transferring Whitelisted Tokens

● LOW

● ACKNOWLEDGED

DESCRIPTION

Several container functions rely on `_validateWhitelistedTokensBeforeReport()`, which reverts if any whitelisted token balance exceeds its dust threshold. An attacker can transfer a small amount of any whitelisted token (dust) directly to the container, causing these functions to revert until the operator updates the dust threshold or removes the dust. Affected call sites include:

- `ContainerPrincipal.sendDepositRequest()`
- `ContainerAgent.reportDeposit()`
- `ContainerAgent.reportWithdrawal()`
- `ContainerLocal.reportDeposit()`
- `ContainerLocal.reportWithdraw()`
- `ContainerPrincipal.reportDeposit()`
- `ContainerPrincipal.reportWithdrawal()`

RECOMMENDATION

We recommend using a pull-based accounting model for expected tokens rather than relying on `balanceOf()` checks, or allow the operator to sweep unexpected token balances before reporting.

CLIENT'S COMMENTARY

At the moment, the current design is intentional and aligned with system constraints. Design choice mitigated by dust threshold configuration.

L-15 Strategy Exit and Enter Uses Pull Pattern for Transferring Tokens, Reducing Composability

● LOW

● ACKNOWLEDGED

DESCRIPTION

`_exitStrategy()` uses `safeTransferFrom(strategy, container, amount)` for each token, requiring the strategy to have pre-approved the container. A push pattern (where the strategy transfers directly) would be simpler and more composable with strategies that manage approvals internally.

Similarly, when `_enterStrategy()` has a remainder after entering a strategy, it loops through all input tokens and calls `safeTransferFrom(strategy, container, remainingAmounts[i])` for each. This pull-based approach requires the strategy to maintain approvals. A simpler design would have the strategy push all remaining tokens back during `enter()`.

RECOMMENDATION

We recommend using a push-based pattern where the strategy pushes tokens to the container during `exit()` and `enter()`, reducing approval management complexity.

CLIENT'S COMMENTARY

At the moment, the current design is intentional and aligned with system constraints. Design choice mitigated by dust threshold configuration.

L-16 CCTPv2 blacklistDomain Does Not Validate domainId Matches Stored Value and Leaks Stale Reverse Mappings

● LOW

● FIXED AS OF 85d41f98

DESCRIPTION

`blacklistDomain(chainId, domainId)` does not verify that the provided `domainId` matches `_domainsByChainId[chainId].domainId`. A governance error could pass a wrong `domainId` and the function would still execute, emitting an incorrect `DomainBlacklisted` event. Additionally, `_chainIdByDomainId[domainId]` is never cleared during blacklisting, leaving stale reverse mappings. While the `handleReceiveFinalizedMessage` checks prevent exploitation (the `domainId == sourceDomain` check would fail for stale mappings), the state remains inconsistent.

RECOMMENDATION

We recommend adding `require(domain.domainId == domainId)` and clearing `_chainIdByDomainId[domainId] = 0` in `blacklistDomain`.

L-17 AcrossBridgeAdapter Validates fillDeadline But Never Uses It in the Bridge Call

● LOW

● FIXED AS OF 59c80e13

DESCRIPTION

`_validatePayload()` checks `acrossParams.fillDeadline >= block.timestamp`, but `fillDeadline` is never passed to `depositV3Now()`. Instead, the `fillDeadlineOffset` parameter uses `ISpookyPool(spookyPool).fillDeadlineBuffer()`. The validation is dead code — it validates a parameter that has no effect on the bridge operation. The operator has no real control over the fill deadline.

RECOMMENDATION

We recommend either passing `acrossParams.fillDeadline` to `depositV3Now()` as intended, or removing the dead validation to avoid confusion.

L-18 UniswapV3Adapter whitelistPath Allows Zero-Hop Paths

● LOW

● FIXED AS OF 3e47d624

DESCRIPTION

Calling `whitelistPath([singleToken], [])` satisfies `tokens.length == fees.length + 1` ($1 == 0 + 1$). The resulting 20-byte path is not a valid Uniswap V3 path (requires at least 43 bytes: address + fee + address). The path would be whitelisted but any swap attempt would revert at the Uniswap router.

RECOMMENDATION

We recommend adding `require(fees.length > 0, "Path must have at least one hop")`.

L-19 Naming and Error Message Clarity

● LOW

● ACKNOWLEDGED

DESCRIPTION

- In `Vault.sol`, `maxDepositBatchSize` and `minDepositBatchSize` indicate notion token amounts, not counts. Rename to `maxDepositBatchAmount` and `minDepositBatchAmount` for clarity.
- In `OneInchAdapter.sol`, replace `InvalidSrcReceiver(descs.srcReceiver)` error with `InvalidDstReceiver(descs.dstReceiver)` in the corresponding `require` for accuracy.

RECOMMENDATION

We recommend updating variable names and error references as described.

CLIENT'S COMMENTARY

MixBytes: Only the error message in `OneInchAdapter.sol` was updated; the Vault batch size naming was not changed.

Client: Renaming variables in this case would require widespread changes across the codebase, including tests, without providing meaningful improvement in readability or maintainability. Given the limited benefit and high refactoring cost, we consider the current naming acceptable.

L-20 Redundant Unchecked Loop Increments

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

The contracts target Solidity ^0.8.28, where the compiler already emits unchecked arithmetic for simple `for` loop counters (optimization introduced in 0.8.22 and enabled by default). In the following locations, the counter is manually incremented inside the loop body using `unchecked { ++i; }` with an empty increment clause in the `for` statement:

- `shift-defi-platform/contracts/StrategyTemplate.sol`:
 - `setInputTokens()`
 - `setOutputTokens()`
 - `emergencyExitMultiple()`
 - `_takeFundsFromAgent()`
 - `_prepareFundsAfterEnter()`
 - `_prepareFundsAfterExit()`
 - `_tokensAmountsDump()`
- `shift-defi-platform/contracts/libraries/helpers/RingCacheLibrary.sol`:
 - `all()`

For each of these loops, the counter is a local `uint256`, incremented only by `++i`, and not modified elsewhere in the loop body. This matches the compiler's auto-unchecked pattern for `for (...; ...; ++i)` in 0.8.28, so the manual `unchecked` is redundant and reduces readability without additional gas savings.

RECOMMENDATION

We recommend switching to a standard `for (...; ...; ++i)` loop and removing the manual `unchecked` increment, unless the loop is intentionally structured to bypass the compiler optimization or the counter is modified elsewhere (in which case, document the reason explicitly).

L-21 Vault claim* Reverts When Nothing to Claim, Breaking Integrations

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

`Vault.claimDeposit` and `Vault.claimWithdraw` revert with `NothingToClaim()` when the requested account has no pending amount (or calculated claim amount is zero). This makes the claim flow non-idempotent for external integrations that expect a "safe" claim call to succeed even when there is nothing to claim. In practice, an integration might submit a claim transaction for a user (or as a batch step) and expect it to be a no-op if the user already claimed; instead it reverts and can break the overall workflow.

This is visible in:

- `Vault.claimDeposit`: `require(vars.depositAmount > 0, NothingToClaim())` and later `require(vars.sharesToClaim > 0 || vars.notionToClaim > 0, NothingToClaim())`.
- `Vault.claimWithdraw`: `require(vars.notionToClaim > 0, NothingToClaim())`.

RECOMMENDATION

We recommend making `claimDeposit` / `claimWithdraw` idempotent by returning early (no-op) when there is nothing to claim, instead of reverting. This improves integration reliability and allows safe repeated calls without breaking batch flows.

L-22 Meaningless Balance-Delta Check After Bridging

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

In `CrossChainContainer._bridgeToken`, the check `balanceBefore - balanceAfter >= bridgedAmount` does not make sense because it only tests a scenario where the adapter spent fewer tokens than required, which is not a realistic failure mode for a correct adapter. As a result, the condition does not add safety but can still introduce confusing behavior.

RECOMMENDATION

We recommend removing this check or replacing it with a validation that matches the actual invariant for each adapter (if needed).

L-23 Too Strict Slippage Check on bridgedAmount

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

In `CrossChainContainer._bridgeToken`, the condition `bridgedAmount > minTokenAmount` is too strict. When the adapter returns exactly `minTokenAmount` (a valid, boundary-case outcome), the call reverts with `BridgeSlippageExceeded` even though slippage is within the user's minimum.

RECOMMENDATION

We recommend using `>=` instead of `>` for the slippage comparison, or otherwise documenting that an exact match is considered a failure.

L-24 Nonce Check Can Break Delivery on Message Reordering

● LOW

● FIXED AS OF 98f9ff21

DESCRIPTION

`MessageRouter.receiveMessage` enforces a monotonic nonce per path. If LayerZero delivers messages out of order, a later nonce can be accepted first and a valid earlier nonce will then revert as a replay, permanently blocking that message. In the current architecture this reordering is unlikely, but the failure mode still exists.

RECOMMENDATION

We recommend preventing reordering at the LayerZero level (enforce ordered delivery / block out-of-order messages) so that the nonce check cannot lock out valid messages.

L-25 No Way to Remove Predefined Swaps

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

`SwapRouter.setPredefinedSwapParameters` can only set (overwrite) parameters but provides no way to delete them. Once a predefined swap is set, it remains active unless replaced with another adapter/payload, which makes it impossible to intentionally remove a predefined swap for a pair.

RECOMMENDATION

We recommend adding an explicit removal mechanism (e.g., a function that clears the adapter to `address(0)` or deletes the mapping entry) to allow disabling a predefined swap when needed.

L-26 Strict Inequalities in Deposit Limits

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

Both `Vault._setMaxDepositAmount` and `Vault._setMinDepositAmount` use strict comparisons, which reject the boundary case where `maxDepositAmount == minDepositAmount`. There is no safety benefit to excluding equality, and it unnecessarily prevents configurations where the min equals the max.

RECOMMENDATION

We recommend using non-strict comparisons (`>=` / `<=`) to allow `minDepositAmount == maxDepositAmount`.

L-27 `setContainerWeights()` Allows Duplicate Container Addresses

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

`Vault.setContainerWeights()` accepts arrays with duplicate container addresses. This can cause the input parameters to be interpreted incorrectly, even though the invariant "sum of weights equals MAX_BPS" is preserved.

RECOMMENDATION

We recommend rejecting duplicate addresses in the input (e.g., enforce uniqueness) to avoid ambiguous or misleading configurations.

L-28 `startDepositBatchProcessing` Can Revert on Zero Container Amount

● LOW

● FIXED AS OF 8392c1b3

DESCRIPTION

`Vault.startDepositBatchProcessing` computes a per-container amount and requires `containerAmount > 0`. In edge cases with low `totalBatchDepositAmount` and certain weight distributions, a container can receive `0`, which triggers `IncorrectContainerAmount` and reverts the batch start.

RECOMMENDATION

We recommend preventing rounding-to-zero allocations during deposit batch distribution by adding explicit validation at the configuration level (e.g., when calling `setContainerWeights` and/or setting `minDepositBatchSize`) to ensure that, for the minimum expected batch size, each non-zero-weight container receives a non-zero `containerAmount`. While a revert in `startDepositBatchProcessing()` can be mitigated operationally by adjusting weights via `Vault.setContainerWeights` (or by increasing `minDepositBatchSize` / waiting for a larger buffered batch), it is preferable to prevent such configurations upfront to avoid operator intervention and potential batch liveness issues.

L-29 Single EMERGENCY_PAUSER_ROLE Controls Both pause() and unpause() With No Threshold Differentiation

● LOW

● ACKNOWLEDGED

DESCRIPTION

Both `pause()` and `unpause()` are gated by the same `EMERGENCY_PAUSER_ROLE` :

```
function pause() external whenNotPaused onlyRole(EMERGENCY_PAUSER_ROLE) {
    _pause();
}

function unpause() external whenPaused onlyRole(EMERGENCY_PAUSER_ROLE) {
    _unpause();
}
```

The role is assigned to a single address at initialization:

```
_grantRole(EMERGENCY_PAUSER_ROLE, roleAddresses.emergencyPauser);
```

This means that whoever holds `EMERGENCY_PAUSER_ROLE` can both freeze all user funds instantly and unfreeze them with no additional oversight. The security properties of pause and unpause are fundamentally different:

- Pause is an emergency action — it must be executable immediately, by as few signers as possible, to respond quickly to an exploit.
- Unpause is a recovery action — it signals that the incident is resolved and normal operations are safe to resume. This requires higher confidence and should demand more approvals.

RECOMMENDATION

We recommend splitting pause and unpause into two separate roles.

CLIENT'S COMMENTARY

The separation of roles for pause and unpause operations is intentional and reflects internal security and operational requirements

L-30 Missing `setConfig()` Wrapper for Library Parameter Tuning

● LOW

● FIXED AS OF fe689dbd

DESCRIPTION

The `ShiftOApp` adapter does not implement a wrapper for the `setConfig()` function, which is used to configure library-specific parameters within the selected send/receive libraries. While the adapter functions correctly with default library parameters, this design choice limits operational flexibility for post-deployment adjustments to critical settings such as DVN requirements, executor parameters, and fee structures.

Currently, if the protocol needs to adjust library parameters (e.g., increase the number of required DVNs from 2 to 4 for enhanced security, or change executor settings), the owner would need to:

1. Call `OAppCore.setDelegate(ownerAddress)` to temporarily grant themselves delegate permissions
2. Call `EndpointV2.setConfig()` directly with library parameters
3. Optionally revert the delegate back to `address(this)` via `OAppCore.setDelegate(address(this))`

Without a `setConfig()` wrapper, the adapter forces a multi-step delegate workaround for what could be a single `onlyOwner` call on the OApp, which adds operational overhead and room for error (e.g., forgetting to restore the delegate).

RECOMMENDATION

We recommend adding a `setConfig()` wrapper function to `ShiftOApp`, allowing the owner to tune library-specific parameters without additional operational steps.

L-31 Missing `_disableInitializers()`

● LOW

● FIXED AS OF 79ca6dc8, a9c48974, 241d6671

DESCRIPTION

Several upgradeable implementations in scope omit the standard implementation-side guard: a constructor that calls `_disableInitializers()`. Examples include `CCTPv2BridgeAdapter`, `AcrossBridgeAdapter`, and `MessageRouter`.

OpenZeppelin's upgradeable contract guidance recommends an empty constructor on the implementation contract:

```
constructor() {  
    _disableInitializers();  
}
```

Without it, the logic contract remains directly initializable, so `initialize` can be called on the implementation address rather than only through the proxy—weak hardening for transparent/UUPS deployments and a footgun for tooling or operators that target the wrong contract.

RECOMMENDATION

We recommend adding a constructor that disables initializers on each upgradeable implementation.

L-32 blacklistDomain Leaves Stale domainId in Domain Storage

● LOW

● FIXED AS OF a9c48974

DESCRIPTION

In `CCTPv2BridgeAdapter.blacklistDomain`, the contract sets `domain.isWhitelisted = false` and clears the reverse mapping `_chainIdByDomainId[domainId]`, but it does not reset `domain.domainId` on the `Domain` struct for that `chainId`.

As a result, storage remains internally inconsistent: the chain is no longer whitelisted (so `_validatePayload` correctly blocks new outbound burns via `isWhitelisted`), but `_domainsByChainId[chainId].domainId` still holds the previous Circle domain id.

The public view `getDomainId(uint256 chainId)` returns `domain.domainId` without checking `isWhitelisted`. After blacklisting, integrators, monitoring, or off-chain tooling that rely on `getDomainId` can still read a non-zero domain id and treat the chain as configured, even though bridging to that domain is disabled.

Clearing `domain.domainId` on blacklist would align the forward and reverse mappings with the semantic "no active domain for this chain" and avoid stale reads.

RECOMMENDATION

We recommend resetting the stored domain identifier on blacklist so mappings and whitelist semantics stay aligned, and either documenting that `getDomainId` must be read together with the whitelist flag or changing `getDomainId` to return `0` when `!domain.isWhitelisted` so the view matches operational reality.

L-33 CustomOracleWrapper.getPrice() — Returns Zero for Unset Tokens Without Reverting

● LOW

● FIXED AS OF 79ca6dc8

DESCRIPTION

`CustomOracleWrapper.getPrice()` returns `(tokenToPrice[token], DEFAULT_PRICE_DECIMALS)` directly from the mapping. For tokens that have never had a price submitted via `submitPrice()`, the mapping defaults to 0, so the function returns `(0, 8)` without reverting. This contrasts with `ChainLinkOracleWrapper` which explicitly validates `require(price > 0, ZeroPrice(token))`.

RECOMMENDATION

We recommend adding a zero-price check in `getPrice()` to revert with a descriptive error when no price has been submitted.

L-34 ChainlinkOracleWrapper — Single Global Staleness Threshold for All Feeds

● LOW

● FIXED AS OF 79ca6dc8

DESCRIPTION

`ChainlinkOracleWrapper` uses a single `priceFeedStalenessThreshold` applied to all token feeds in `_getAndValidateLatestRoundData()`. Chainlink documents a heartbeat per feed/asset, so staleness limits should be chosen per token to match that feed—not one global value for every asset.

RECOMMENDATION

We recommend implementing per-token staleness thresholds via a `mapping(address => uint256)` and setting the threshold alongside the feed in `setChainlinkFeed()`.

L-35 ChainlinkOracleWrapper — No L2 Sequencer Uptime Check

● LOW

● ACKNOWLEDGED

DESCRIPTION

`ChainlinkOracleWrapper._getAndValidateLatestRoundData()` does not check the L2 sequencer uptime feed before using price data. On L2 chains, when the sequencer goes down and comes back up, Chainlink feeds may report stale pre-downtime prices that pass the staleness check because `updatedAt` is relative to L2 block time, which was frozen during downtime. Chainlink recommends checking the sequencer uptime feed on all L2 deployments to ensure that price data is not consumed immediately after a sequencer restart, before feeds have had time to update.

RECOMMENDATION

We recommend adding an L2 sequencer uptime feed check with a grace period, following Chainlink's recommended pattern: <https://docs.chain.link/data-feeds/l2-sequencer-feeds>

CLIENT'S COMMENTARY

While monitoring the L2 sequencer status in addition to price staleness is a valid recommendation, we have assessed the risk as acceptable within our use case

L-36 PriceOracleAggregator — Single Oracle per Token With No Fallback

● LOW

● ACKNOWLEDGED

DESCRIPTION

The protocol routes each token through a single registered oracle:

`PriceOracleAggregator.priceOracles` stores one implementation per token, and there is no secondary or fallback oracle. Both `fetchTokenPrice()` and `getRelativeValueUnified()` call `IPriceOracle.getPrice()` on that address with no `try / catch` and no alternate source.

If the configured oracle reverts (e.g. Chainlink feed stale, paused, or misconfigured), any call to `fetchTokenPrice()` or `getRelativeValueUnified()` fails, and every upstream function that depends on those entry points reverts with it.

RECOMMENDATION

We recommend using a fallback oracle per token: register a secondary price source and, when the primary oracle's `getPrice` reverts or returns unusable data, call the fallback instead. The same primary/fallback pattern should apply in both `fetchTokenPrice()` and `getRelativeValueUnified()`.

CLIENT'S COMMENTARY

The absence of a fallback oracle is intentional and aligned with the protocol design.

L-37 Container, CrossChainContainer, StrategyTemplate — Missing Storage Gaps (`__gap`) in Base Abstract Contracts

● LOW

● FIXED AS OF 79ca6dc8

DESCRIPTION

The abstract base contracts `Container.sol`, `CrossChainContainer.sol`, and `StrategyTemplate.sol` are inherited by multiple upgradeable concrete contracts but do not declare `uint256[N] private __gap` storage reservations. While `StrategyContainer` and `BridgeAdapter` correctly have `__gap[50]`, the gap in `StrategyContainer` does NOT protect against additions in `Container` because `Container`'s storage comes before `StrategyContainer` in the C3 linearization. If a future upgrade adds a state variable to `Container`, it will shift the storage layout of all derived contracts (`ContainerLocal`, `ContainerPrincipal`, `ContainerAgent`), corrupting their state (including critical fields like `vault`, `notion`, `_whitelistedTokens`).

RECOMMENDATION

We recommend either adding `uint256[50] private __gap` at the end of `Container.sol`, `CrossChainContainer.sol`, and `StrategyTemplate.sol`, or adopting a namespaced storage layout (e.g. ERC-7201-style `keccak256` storage slots) so future layout changes cannot collide with derived contracts' state.

L-38 `CrossChainContainer.setBridgeAdapter()` and `ReshufflingGateway.blacklistBridgeAdapter()` — De-Listed Adapters Retain Token Approvals

● LOW

● ACKNOWLEDGED

DESCRIPTION

Two contracts grant ERC-20 approvals to bridge adapters on a per-call basis but never revoke them when an adapter is removed:

- **CrossChainContainer** : `_approveTokenToBridgeAdapter()` calls `forceApprove(bridgeAdapter, amount)` before each bridge operation. When the adapter is de-supported via `setBridgeAdapter(adapter, false)`, the approval is not revoked.
- **ReshufflingGateway** : `bridgeTokens()` calls `safeIncreaseAllowance(bridgeAdapters[i], instructions[i].amount)` before each bridge call, so allowances accumulate across transactions. When the adapter is blacklisted via `blacklistBridgeAdapter()`, the accumulated approval is not revoked.

RECOMMENDATION

We recommend making adapter de-listing consistent across the protocol: some code paths may already revoke outstanding ERC-20 allowances when an adapter is removed, while `CrossChainContainer.setBridgeAdapter(adapter, false)` and `ReshufflingGateway.blacklistBridgeAdapter()` do not. The same explicit revoke step (e.g. `forceApprove(adapter, 0)` for every relevant token) should apply everywhere an adapter is disabled, so behaviour is uniform rather than mixed.

CLIENT'S COMMENTARY

It is impossible to maintain entire list of whitelisted tokens on bridge adapter, since the list is subject to modification

L-39 StrategyContainer.isStrategyNavUnresolved() — Incorrect Result for Non-Existent Strategies

● LOW

● FIXED AS OF 79ca6dc8

DESCRIPTION

`StrategyContainer.isStrategyNavUnresolved()` discards the `found` boolean from `_strategies.indexOf(strategy)`. For non-existent strategies, `indexOf()` returns `(false, 0)`, and the function checks bit 0 of the bitmask — corresponding to a different strategy. All internal callers validate membership first, but the function is `public` and can return misleading results to external consumers.

RECOMMENDATION

We recommend either adding a membership check (`if (!found) return false`) or changing visibility to `internal`.

L-40 Vault.claimWithdraw() — Shares Permanently Locked When Notion Rounds to Zero

● LOW

● FIXED AS OF 79ca6dc8

DESCRIPTION

In `Vault.claimWithdraw()`, `notionToClaim` is computed via `mulDiv`:

```
vars.notionToClaim = vars.withdrawnShares.mulDiv(
    withdrawBatchTotalNotion[batchId],
    withdrawBatchTotalShares[batchId]
);
if (vars.notionToClaim == 0) {
    return 0;
}
```

When `withdrawnShares` is very small relative to `withdrawBatchTotalShares`, the result rounds down to zero. The function returns early without clearing `pendingBatchWithdrawals[batchId]` `[onBehalfOf]` and without calling `_burn()`. The user's shares remain on the Vault's balance permanently — repeated calls always return 0 since the same rounding applies, so there is no recovery path for the affected user.

RECOMMENDATION

We recommend separating the "no pending withdrawal" check (`withdrawnShares == 0`) from the "notion rounds to zero" case. When `withdrawnShares > 0`, always clear the record and burn the shares regardless of whether `notionToClaim` is zero. This way shares are always freed, even if the corresponding notion payout is dust.

L-41 Empty DepositRequest Leaves ContainerAgent in Unrecoverable DepositRequestReceived

● LOW

● FIXED AS OF 79ca6dc8

DESCRIPTION

A deposit request with `tokens.length == 0` leaves `claimCounter` at zero but still sets `DepositRequestReceived`. `claim()` then always reverts in `_claimExpectedToken` (`require(claimCounter > 0)`), so the agent cannot complete the deposit / Vault report flow. The live Principal stack cannot produce this payload, the gap is mainly defense in depth for future changes.

RECOMMENDATION

We recommend ensuring `request.tokens.length > 0` before `DepositRequestReceived`.

L-42 Strict Slippage Inequality in `_enterToState()` Reverts on Exact `minNavDelta` Match

● LOW

● FIXED AS OF 79ca6dc8

DESCRIPTION

The private `StrategyTemplate._enterToState()` path (re-enter to a state) applies the same class of slippage check as the historical Low #9 issue in `_enter()`, but still uses a **strict** inequality.

After `_enterTarget()` / `_enterState(toStateId)`:

```
vars.stateToNavAfterEnter = stateNav(toStateId);  
  
require(vars.stateToNavAfterEnter > vars.stateToNavBeforeEnter + minNavDelta);
```

In `_enter()`, the analogous check was corrected to **non-strict** `>=`, so an NAV increase exactly equal to `minNavDelta` succeeds there. In `_enterToState()`, an exact match still reverts. That is inconsistent between code paths and forces integrators to set `minNavDelta` slightly below the true floor to avoid spurious failures at the boundary.

RECOMMENDATION

We recommend replacing `>` with `>=` in `_enterToState()` so `minNavDelta` is inclusive, matching `_enter()` and standard slippage semantics. If strictness is intentional for re-enters only, document it clearly at the API level.

L-43 `defaultPriceFeedStalenessThreshold` Has No Post-Construction Setter in `ChainlinkOracleWrapper`

● LOW

● FIXED AS OF c9d848e1

DESCRIPTION

The new `defaultPriceFeedStalenessThreshold` is set only in the constructor; no setter is exposed. Operators (`ORACLE_MANAGER_ROLE`) can override per-token via `setPriceFeedStalenessThreshold(token, threshold)` , but cannot retune the default. Because `ChainlinkOracleWrapper` is non-upgradable, the default is effectively immutable for the lifetime of the deployment — adjusting it requires a full redeploy and re-wire through `PriceOracleAggregator` .

RECOMMENDATION

We recommend adding a setter restricted to `ORACLE_MANAGER_ROLE` .

L-44 StrategyTemplate.emergencyExit() — Failure Handling Is Unreachable Unless emergencyExitMaxSlippage == MAX_BPS

● LOW

● FIXED AS OF 30f27218

DESCRIPTION

`StrategyTemplate.emergencyExit()` self-calls `tryEmergencyExit()` and catches an adapter revert into `isExitSuccess`, so the strategy can be parked in NAV-resolution mode (emitting `EmergencyExitFailed`) instead of reverting. However, the destination-NAV slippage gate runs unconditionally **before** that branch is evaluated:

```
(vars.isExitSuccess, returnData) = address(this).call(
    abi.encodeWithSelector(this.tryEmergencyExit.selector, toStateId, share)
);

vars.toStateNavAfterExit = stateNav(toStateId);
require(
    vars.toStateNavAfterExit >= vars.toStateNavBeforeExit + minNavDelta,
    SlippageCheckFailed(vars.toStateNavBeforeExit, vars.toStateNavAfterExit,
minNavDelta)
);

if (vars.isExitSuccess) {
    if (share == MAX_BPS) { _acceptNav(toStateId); }
    emit EmergencyExitSucceeded(toStateId);
} else {
    emit EmergencyExitFailed(toStateId);
}
```

On a caught adapter failure no funds move, so `toStateNavAfterExit == toStateNavBeforeExit`, and the `require` passes only when `minNavDelta == 0`. The entry gate admits `minNavDelta == 0` only when `emergencyExitMaxSlippage == MAX_BPS`. Therefore, under any slippage-protected configuration (`emergencyExitMaxSlippage < MAX_BPS`), a genuine adapter `redeem` revert makes the **entire** `emergencyExit()` revert — rolling back `startEmergencyResolution()` and the resolution flag — instead of reaching the failure-handling branch the catch exists to serve. The operator cannot park the strategy / mark its NAV unresolved during the exact incident the mechanism is intended for; the only workaround is to widen the cap to `MAX_BPS` and retry, which simultaneously disables the destination-NAV floor.

RECOMMENDATION

We recommend evaluating `isExitSuccess` before enforcing the destination-NAV floor: apply the floor only on the success branch, and on the caught-failure branch skip it, emit `EmergencyExitFailed`, and remain in resolution mode — for any configured `emergencyExitMaxSlippage`.

L-45 StrategyTemplate — Dangling Router Allowance After a No-Route Swap

● LOW

● FIXED AS OF 30f27218

DESCRIPTION

`_swapToInputTokens()` calls `forceApprove(swapRouter, amountIn)` and then `tryPredefinedSwap(...)` with `mustSucceed = false`. If no route exists the swap pulls nothing, leaving a non-zero `tokenIn → swapRouter` allowance dangling until the next successful swap of that token.

RECOMMENDATION

We recommend resetting the router allowance to zero (`forceApprove(swapRouter, 0)`) after the swap.

L-46 ShiftOApp Ordered Delivery Omits the skipInboundNonce Recovery Helper

● LOW

● FIXED AS OF a6ed2b5c

DESCRIPTION

Commit `c28061e` opts `ShiftOApp` into LayerZero ordered delivery (`nextNonce = receivedNonce + 1` , enforced by `_acceptNonce`), matching the canonical LZ example but omitting its `skipInboundNonce` helper. Under ordered delivery a message that reverts at the head of the queue stalls the whole shared (`srcEid` , `sender`) channel (all Containers) until it is resolved, and the synchronous receive path (`_lzReceive` → `MessageRouter.receiveMessage` → `ContainerPrincipal` / `ContainerAgent.receiveMessage`) reverts when the target Container is paused, in emergency/reshuffling, or off-status.

These stalls are recoverable. The Endpoint keeps the payload verified, so a transient revert (pause / emergency / reshuffling) self-heals once the condition clears and the message is retried — no intervention. A would-be permanent revert (a malformed payload or an unrecoverable status) is cleared by upgrading the receiving `MessageRouter` / Container — both upgradeable — so the message processes and the nonce advances. The only gap is that `endpoint.skip` alone does not advance the app's `receivedNonce` (per the LZ "Keeping nonces in sync" docs), so the lighter `skipInboundNonce` recovery the LZ example ships is unavailable here.

RECOMMENDATION

We recommend adding `skipInboundNonce(srcEid, sender, nonce)` as in the LZ example — calling `endpoint.skip(...)` and advancing `receivedNonce` in the same call (per LayerZero, "Message Ordering — Keeping nonces in sync": <https://docs.layerzero.network/v2/concepts/message-ordering#keeping-nonces-in-sync>) — or making the receive path non-blocking, so a stalled message can be cleared without an upgrade.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[!\[\]\(a8ff699ced33317c53c86f9bf3171905_img.jpg\)](#) [!\[\]\(7c08f656893b221194f2f1726f0b91e6_img.jpg\)](#) [!\[\]\(e4c419dad7fb4b2eb7c1b5471321c3b4_img.jpg\)](#) [!\[\]\(a883f2d029f637fd11b573676579513e_img.jpg\)](#)